

Algorithm design, problem solving and programming

Paper 2

Zain Merchant

The use of algorithms

Consider what is meant by an algorithm. If you search for algorithm then you will find answers such as a process, or a set of rules, that are followed. This does not mean they are limited to computers. You can find algorithms in all areas of life. You will have followed many of these without even considering that they are algorithms. For example, in mathematics you will have followed a series of steps to solve a problem.

Algorithms can take many forms, and use a variety of words or structures.

Computers follow algorithms. Programs are written as a series of instructions that the computer then follows. Algorithms for software might be millions of lines long, and they have to make use of specific instructions to tell the processor what to do.

A computer can only run machine code, that is binary. If you enter a command into the computer, this has to be converted into binary for the processor to execute (run) it.

Program development life cycle

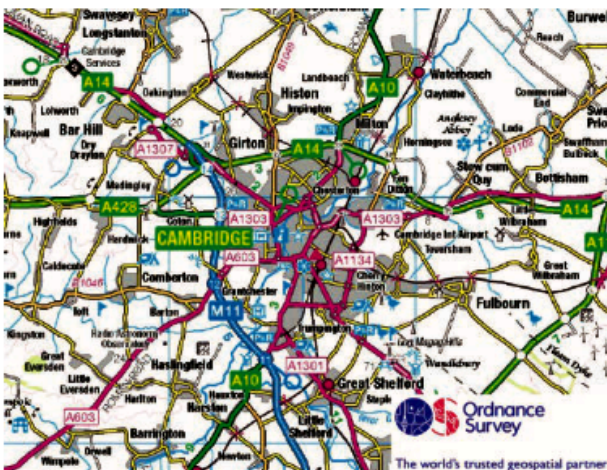
When a person, or organisation, creates a new computer program they will use a structured, organised plan of how to create the program. This is called the program development life cycle.

There are several varieties of life cycle, including cyclic, spiral, waterfall and rapid development. These all include the stages analysis, design, coding and testing.

Analysis

This first stage involves looking at the problem and the system that is needed. The problem is explored, and the requirements of the program are identified. Decomposition is used to split the problem into subproblems. Subproblems allow the developers to identify the requirements for each component part, and then tackle these separately in the next stage. Once you have decomposed the problem, each of the subproblems can be implemented as a subprogram.

Abstraction keeps the key elements required for the solution to the problem and discards any unnecessary details and information that is not required. For example, a map only shows what is required for traveling from one place to another. Different methods of transport will require different types of map.



Design

Once the requirements have been identified, the program designers can begin planning how the program will work in the design phase. This can include an overview of the program using a structure diagram, and the designing of algorithms using flowcharts and pseudocode.

Coding

Once you have decomposed your problem into subproblems, and you have designed the algorithms using flowcharts and/or pseudocode then you can start writing the program in your chosen programming language. This is often referred to as coding.

This will also include some iterative testing. This is testing that is carried out while the program is being developed, for example, if you write the program code for one of the subproblems, then you will test it with a range of different data to make sure it is fully working before moving onto the next step.

Testing

When you have finished your program, you need to carry out testing to make sure it:

- fully works
- does not crash
- meets all requirements

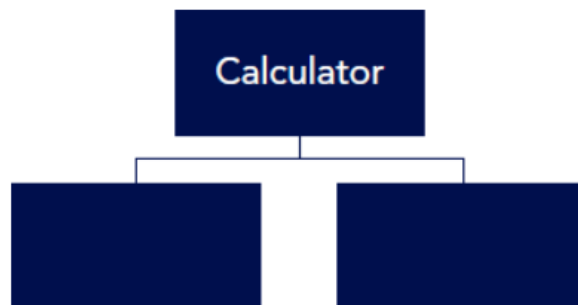
Structure diagram

A structure diagram is developed by decomposing a program into its subprograms. The diagram has the name of the program at the top, and below this its subprograms. Each subprogram can be split down further as well if required.

Example 1

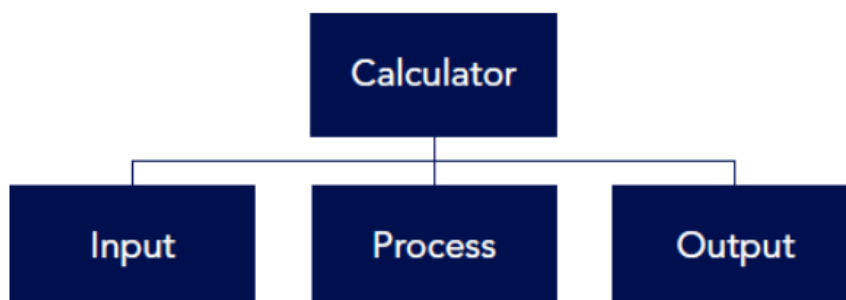
A calculator needs to allow the user to enter numbers and the symbol, calculate the result and output this.

The name of the program is at the top. The boxes below are ready for subprograms

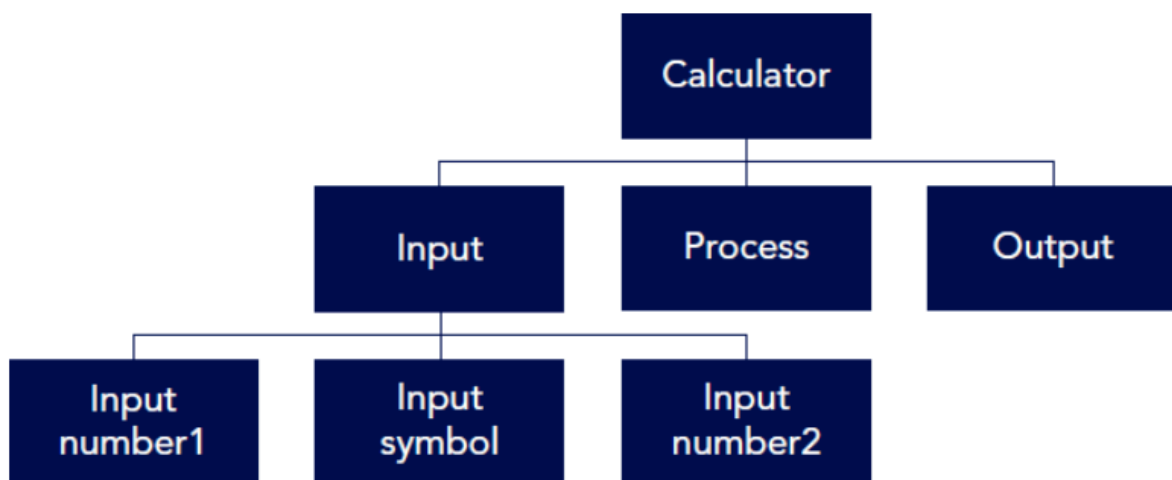


One way of splitting up a small program is to think of inputs, processes and outputs. A calculator

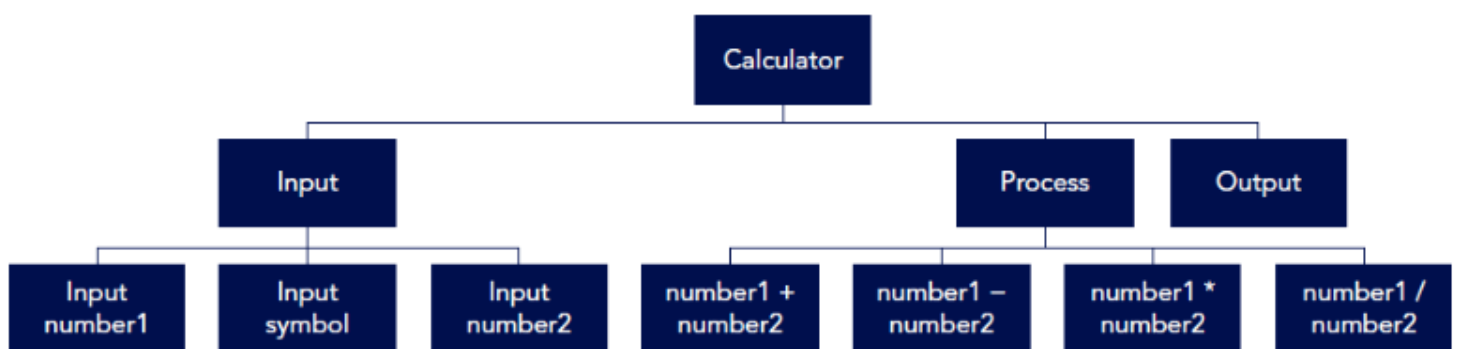
has an input, processes and output, so these can be added to the diagram



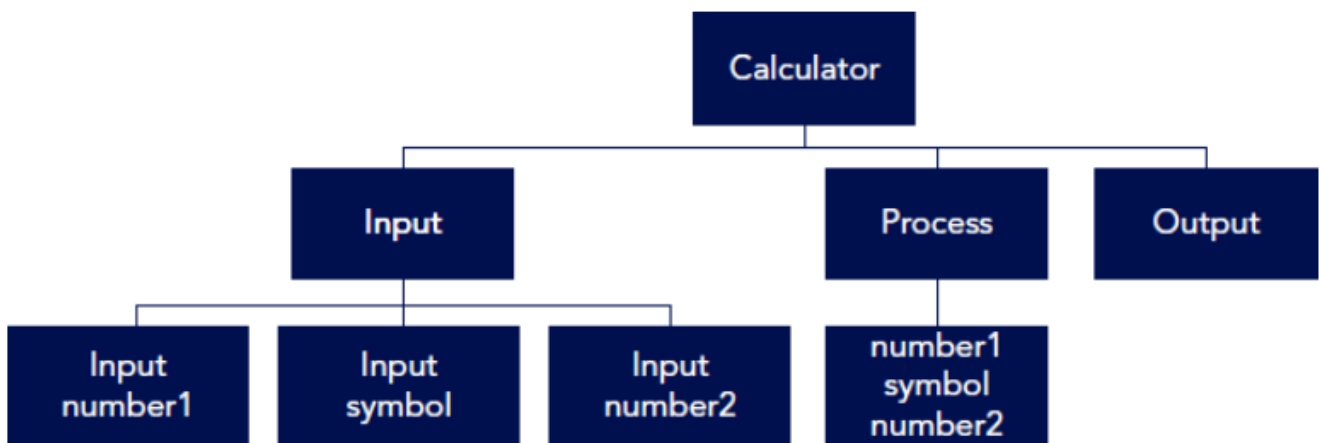
The subprograms can then be split further. The calculator needs two numbers and a symbol to be entered (e.g. $2 + 3$). Add these boxes below input



The process. is dependent on the symbol entered. In this calculator, there is $+$, $-$, $*$ and $/$. Each of these could be given a separate box

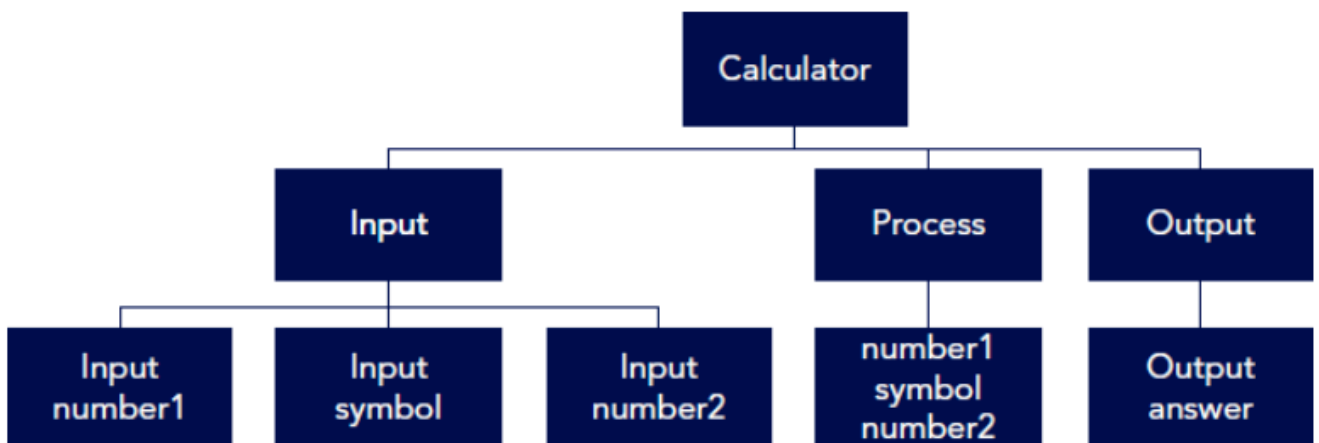


Or the variable names from the input can be used



Both of these are valid. There are lots of ways you can decompose a problem, and everyone could do it slightly differently but the solution is still valid.

Finally, the output needs identifying

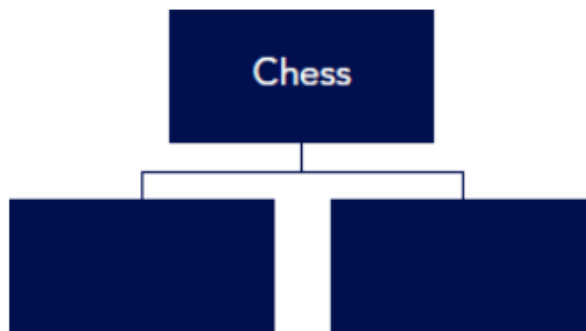


This program was fairly small, so it has been split into very precise instructions that can then be programmed. The next example is a larger problem.

A computerised version of chess is being developed. Two players can enter their names and then a new board will be displayed. The players take it in turns to select pieces to move and the positions to move them to. The program will check that the moves are valid and then move the pieces. It will remove any pieces that have been taken in each move, and check if anyone has reached checkmate.

Example 2

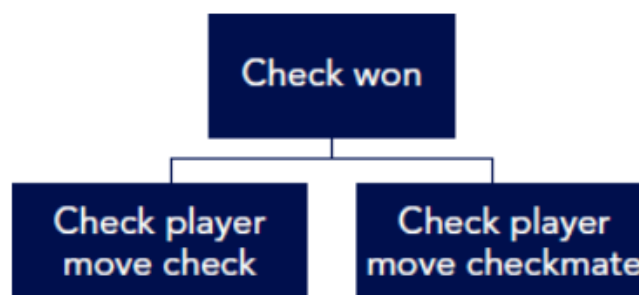
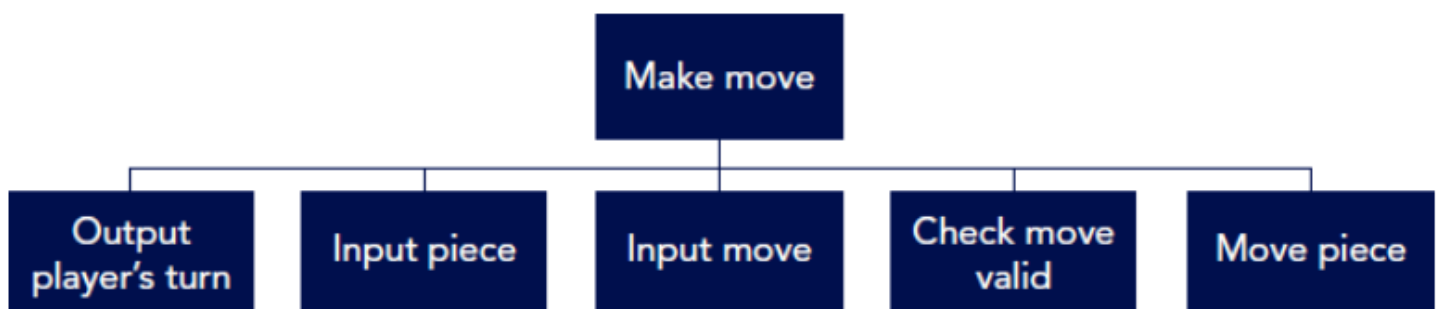
The name of the program is at the top



The program could be decomposed into inputs, processes and outputs, but this is a large program so it can be split down by different functions instead. The first part is that there are two players who need entering, this is given the name 'New players'. Then the game is set up ('New game'). The players can then move their pieces ('Make move'). The program will check if someone has checkmate ('Check won')



These can then be split further. They are each given their own sub-diagram here so that they are easier to view



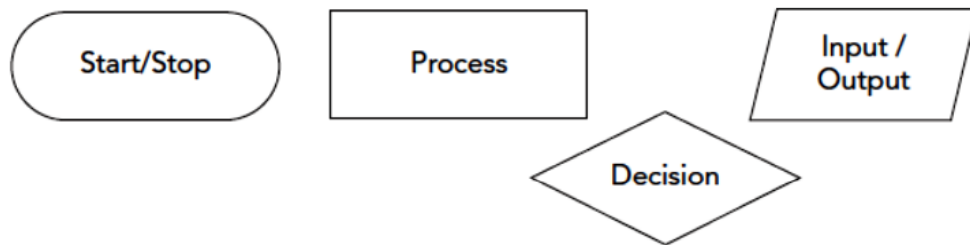
Zain Merchant

- [illegible]

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Flowcharts

A flowchart shows diagrammatically the steps required to complete a task and the order that they are to be performed. These steps, together with the order, are called an algorithm. Flowcharts are an effective way to communicate how the algorithm that makes up a system or sub-system works.

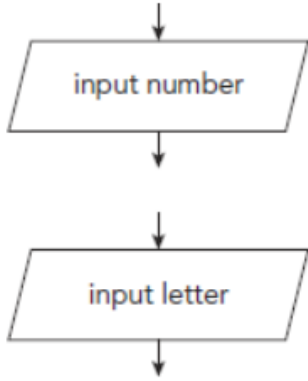


Start/stop

Description	Example
A flowchart begins with a Start box, with one arrow from it. A flowchart ends with a Stop box, with one arrow going into it.	<pre>graph TD; Start([Start]) --> Stop([Stop]);</pre>

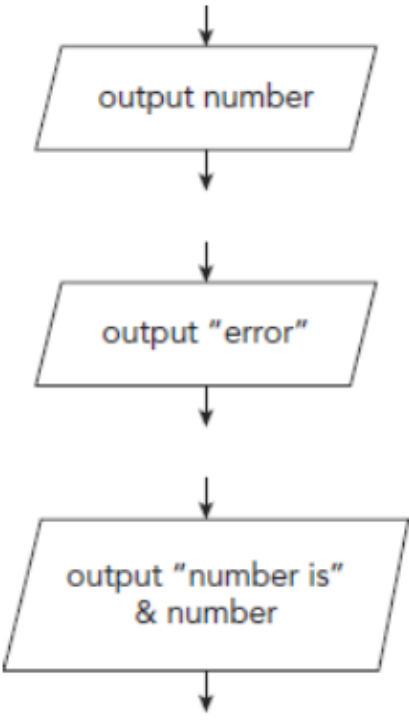
Flowcharts

Input

Description	Example
A single input of a value with an identifier for that value. One arrow should go into the box, and one arrow should come out.	

Zain Merchant

Output

Description	Example
An output of text in speech marks, and/or the identifier of a value	

Process

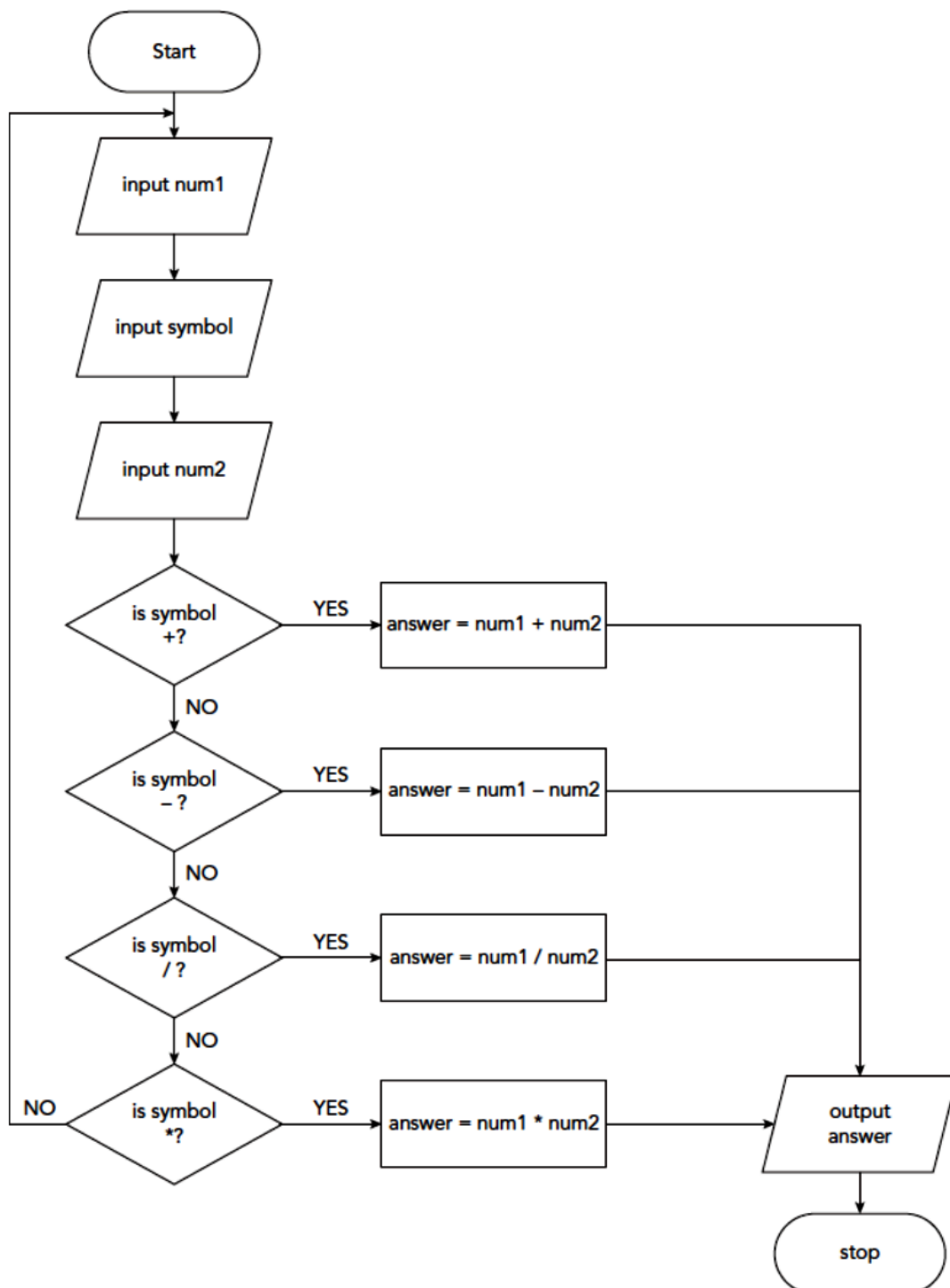
Description	Example
An action that is being performed, usually a mathematical calculation. One arrow goes into the box, and one arrow comes out of the box.	<pre> graph TD Start(()) --> Process1[number = number + 1] Process1 --> Process2[message = "hello" & "world"] Process2 --> End(()) </pre>

Decision

Description	Example
A question with two outcomes, yes or no (or true or false). The question is a comparison from: = equal to > greater than < less than >= greater than <= less than. You can have more than one condition by including AND or OR. A question with two outcomes, yes or no (or true or false). The question is a comparison from: = equal to > greater than < less than >= greater than <= less than.	<pre> graph TD Start1(()) --> Decision1{is 1 > 2?} Decision1 -- YES --> Exit1(()) Decision1 -- NO --> Exit2(()) Start2(()) --> Decision2{is x = y?} Decision2 -- NO --> Exit3(()) Decision2 -- YES --> Exit4(()) </pre>

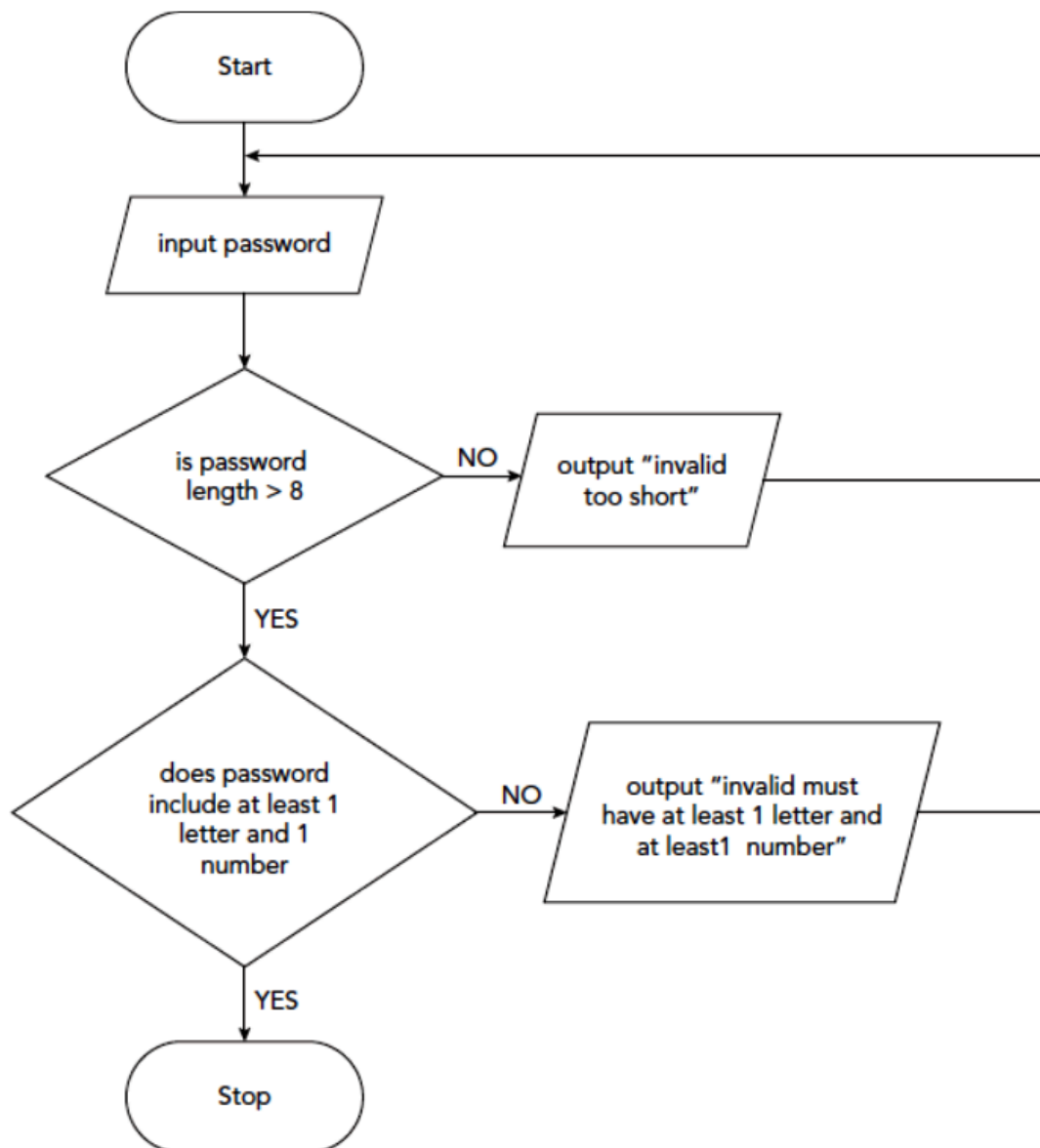
Example 1

A flowchart for a calculator.



Example 2

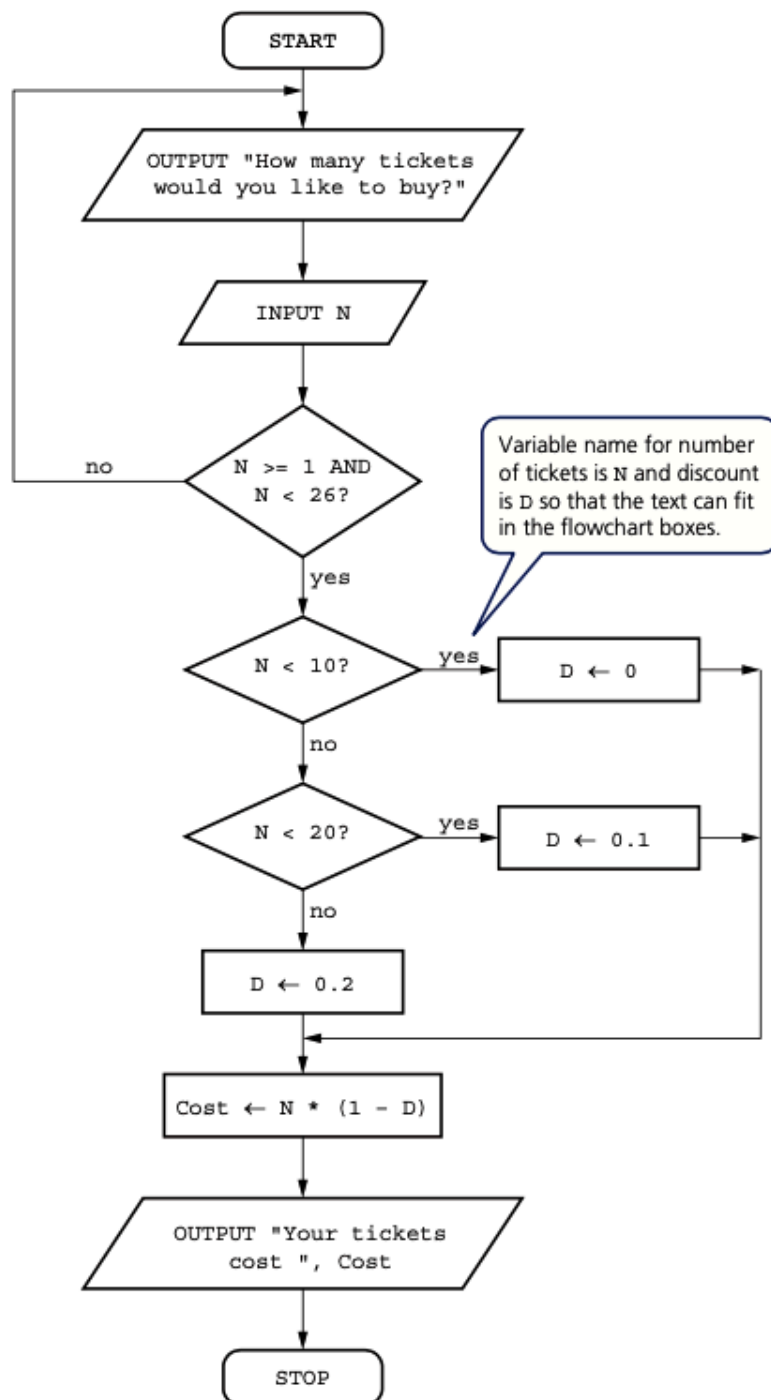
A flowchart for a password validation program.



Example 3

Tickets are sold for a concert at \$20 each, if 10 tickets are bought then the discount is 10%, if 20 tickets are bought the discount is 20%. No more than 25 tickets can be bought in a single transaction.

This is flowchart showing an algorithm to calculate the cost of buying a given number of tickets:



Checkpoint

1. Write the name of each flowchart symbol in the shape.

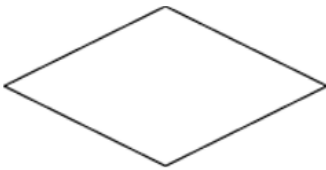
a



b



c



d



2. Draw the flowchart symbol for each statement:

- Start
- INPUT x
- INPUT y d IS $x > y$?
- $x = x + y$
- Stop

3. Draw a flowchart to take three numbers as input, find and output the largest number.

[illegible]

Pseudocode

Pseudocode refers to any code that is not written on a computer to run. There is no one set pseudocode, because if there was then this would be code with a specific language. Instead it's a term for any readable code-like statements that all programmers will understand. This means that it uses keywords, and constructs such as IF statements, WHILE loops, etc. These constructs occur in almost all languages, so any programmer would be expected to read them and translate them into the actual programming language they are using.

Example pseudocode:

```
INPUT Number1  
IF Number1 < 10 THEN  
    OUTPUT "too small"  
ELSE  
    OUTPUT "valid"  
ENDIF
```

This uses the keywords INPUT, IF and OUTPUT. These can then be translated into whatever language the programmer is using.

There are many different valid pseudocode formats. Some will always use capitals for the commands, e.g. INPUT, IF, FOR. Some will use $\leftarrow$ instead of an = for assignment. If you write your pseudocode in almost perfect 'code', i.e. you actually write Python code, because it is not to be run and you can have all sorts of syntax errors it is still pseudocode. What is important is that the program is split into steps that can be easily converted into a real program.

Testing

You will need to identify appropriate test data to use to test the program. There are four types of test data:

- Normal – data that the program should accept.
- Abnormal– data that the program should not accept.
- Extreme– data that is at the edge of what is allowed.
- Boundary– data that is on the edge of being accepted and being rejected.

Example 1

A program allows the user to enter their age as a whole number between 10 and 100 years.

Test data type	Example data
Normal	Any number between 10 and 100 inclusive, e.g. 10, 20, 50, 100, etc.
Abnormal	Any data that is not a number between 10 and 100, e.g. 9, -1, 101, 200, “age”, “30”, 12.2.
Extreme	The largest number accepted – 100. The smallest number accepted – 10.
Boundary	Either side of the largest bound – 100 and 101. Either side of the smallest bound – 9 and 10.

Example 2

A user needs to enter a password at least 8 characters long.

Test data type	Example data
Normal	Any set of 8 or more characters, but they must be text (have speech marks), e.g. "abcdefgh" "12fghj567812".
Abnormal	Any set of characters with 7 or less characters, e.g. "1234567", "seven".
Extreme	Any with 8 characters only, e.g. "12345678", "ajrkdhJK".
Boundary	Either side of the 8 character bound, 7 characters or 8 characters, e.g. "1234567", "12345678", "abcdefg", "iJHFD7hJ",

Checkpoint

1. Identify **three** different types of test data.
2. Why can you have the same test data as normal and extreme types of test data?
3. What is the difference between extreme and boundary test data?
4. A program asks the user to input a number between 1 and 10 (inclusive). Identify test data for each of the test data types in the table:

Test data type	Example data
Normal	
Abnormal	
Extreme	
Boundary	

[illegible]

Programming concepts

Variables and constants

What are variables and constants?

When you are writing a program, you will need to store data; whether this is data that has been input, the result of calculations or any for any other reason. You can store data in variables and constants.

A computer has memory, e.g. RAM. This is made of lots of spaces where you can put data. Imagine this table is memory. Each of the boxes can store a piece of data.

In memory location 0 is the data 10. In memory location 1 is the data red.

0	10
1	Red
2	22.5
3	True
4	
5	
6	
7	

Each variable and constant is one of these spaces in memory that is given an identifier (it has a name). In this table the memory numbers have been replaced with their identifiers. The memory space with the name `number1` is storing the piece of data `10`.

<code>number1</code>	<code>10</code>
<code>colour</code>	<code>red</code>
<code>price</code>	<code>22.5</code>
<code>finished</code>	<code>True</code>
<code>4</code>	
<code>5</code>	
<code>6</code>	
<code>7</code>	

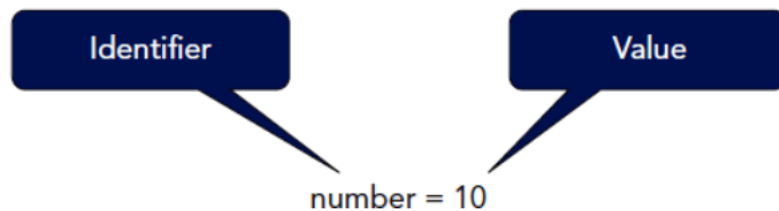
Variables and constants have one difference. In a variable, you can change the data while the program is running. For example, putting the number `30` into memory location `number1`, the memory would then look like this:

<code>number1</code>	<code>30</code>
<code>colour</code>	<code>red</code>
<code>price</code>	<code>22.5</code>
<code>finished</code>	<code>True</code>
<code>4</code>	
<code>5</code>	
<code>6</code>	
<code>7</code>	

A constant cannot have its value changed while the program is running. When you declare a constant you put a value into it and this cannot be changed.

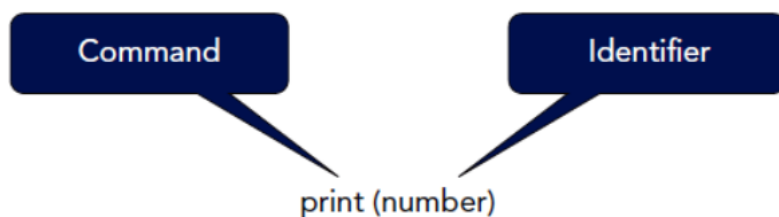
Using variables

Putting data into a variable is done using an assignment statement. The left hand side of the equals sign is the identifier. The right hand side of the equals sign is the value



```
Number ← 10
Colour ← "red"
Price ← 22.2
```

To get data out of a variable you just use its identifier



```
Number ← 10
Colour ← "red"
OUTPUT(Number)
OUTPUT("The colour is ", Colour)
Price ← Number * 2
```

Using constants

Before you use a constant in your program you need to give it a value. This is an assignment statement the same as a variable. (No examples are given for Python as it does not have in-built constants.)

```
CONSTANT Colour ← "yellow"
```

Data types

Data in programs can be of different types. For example, it could be numeric or text. You will need to tell your program what type of data you want it to store. Some programming languages need you to declare what type of data your variable will store when you first use it.

Data type	Description	Example data
STRING	Text – characters numbers and symbols. The data will always need to be inside speech marks, either ' ' or " ".	"hello" "123" "help!"
INTEGER	Whole numbers.	1 23 -300
REAL	Decimal numbers.	1.2 23.0 -20.49
BOOLEAN	Either true or false.	TRUE FALSE
CHAR	One character or number of symbol. The data will always need to be inside speech marks, either " " or "".	h' '9' '?

Storing different data types

Storing a string in a variable:

```
Colour ← "red"
```

Storing an integer in a constant:

```
CONSTANT Value ← 10
```

Storing a real number in a variable:

```
Price ← 22.4
```

Storing a Boolean in a variable:

```
Flag ← TRUE
```

Converting between data types

You might need to turn one data type into another data type. This is not required as part of the specification, but when you are programming in your chosen language you might have to do it for your program to work. This is called casting. You can do this by using the name of the data type you want the data to become.

Example 1

Convert a string to an integer:

```
Number ← int("123")
```

Example 2

Convert a number to a string:

```
Value ← string(22.4)
```

Zain Merchant

- | Statement | Variable | Constant |
|--|----------|----------|
| You cannot change the value when the program is running. | | |
| It has an identifier. | | |
| It is a memory location. | | |
| You can change its value when the program is running. | | |
| It stores a piece of data. | | |

- [illegible]

Output

A program at some point will probably need to give information to the user. It does this using output. When outputting strings (characters, letters, etc.).

Example 1

Output the words, Hello World:

```
OUTPUT("Hello world")
```

Example 2

Output the number 20:

```
OUTPUT(20)
```

If you want to output more than one piece of data then you can join them using a concatenation symbol. Concatenation means join together, so it joins multiple pieces of data together. This could be a comma (,), an ampersand (&) or a plus (+) depending on your language. All are acceptable in pseudocode.

Example 1

Output the word Hello, then the contents of variable name:

```
OUTPUT("Hello ", Name)
```

Example 2

Output the cost of an item stored in the variable cost:

```
OUTPUT("The cost is " , Cost)
```

Example 3

Output the number of balloons stored in the variable balloon:

```
OUTPUT("There are " , Balloon , " balloons")
```

Input

A program might need the user to enter (input) some data. To do this, the command word INPUT is used. This cannot appear on its own, otherwise the data entered will disappear into space. So you need to do something with it, for example, store it in a variable.

Example 1

Input a number and store it in a variable:

```
INPUT Number
```

Example 2

Tell the user to enter a word and store it in a variable:

```
OUTPUT("Enter a word")
```

```
INPUT Word
```

Arithmetic operators

Operator	Description	Example
+	Adds two values together.	$10 + 2$ gives 12 $11.3 + 9$ gives 20.3
-	Subtracts the second value from the first.	$10 - 2$ gives 8 $11.3 - 9$ gives 2.3
*	Multiplies two values together.	$10 * 2$ gives 20 $11.3 * 9$ gives 101.7
/	Divides the first number by the second.	$10 / 2$ gives 5 $11.3 / 9$ gives 1.256
DIV	Gives the whole number after the first number is divided by the second, i.e. it ignores any decimals.	$\text{DIV}(10, 2)$ gives 5 $\text{DIV}(11, 9)$ gives 1
MOD	Gives the remainder after the first number is divided by the second, i.e. how many are left.	$\text{MOD}(10, 2)$ gives 0 $\text{MOD}(11, 9)$ gives 2
^	Power of.	$2^3 = 8$ $3^2 = 9$

MOD has one special use in programming. It can be used to work out if a number is odd or even. If you perform MOD 2 to a number and it returns 0 then it is even, if it returns 1 then it is odd.

Example: $\text{MOD}(10, 2) = 0$ therefore 10 is even. $\text{MOD}(11, 2) = 1$ therefore 11 is odd.

Checkpoint

1. Calculate the result for each of these equations:

- $\text{DIV}(9, 2)$

- $\text{DIV}(17, 3)$

2. Calculate the result for each of these equations:

- $\text{MOD}(9, 2)$

- $\text{MOD}(17, 3)$

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Example 1

Taking two numbers as input and adding them together:

```
OUTPUT("Enter the first number")
```

```
INPUT Num1
```

```
OUTPUT("Enter the second number")
```

```
INPUT Num2
```

```
Total  $\leftarrow$  Num1 + Num2
```

Example 2

Subtracting 10 from 20:

```
Number1  $\leftarrow$  10
```

```
Number2  $\leftarrow$  20
```

```
Result  $\leftarrow$  Number2 - Number1
```

Example 3

Multiplying two values together:

```
Number1  $\leftarrow$  5
```

```
OUTPUT("Enter a number")
```

```
INPUT Number2
```

```
Result  $\leftarrow$  Number1 * Number2
```

Example 4

Dividing 100 by 5:

```
Number1  $\leftarrow$  100
```

```
Number2  $\leftarrow$  5
```

```
Result  $\leftarrow$  Number1 / Number2
```

Example 5

Finding the whole number after dividing 33 by 7:

Result $\leftarrow \text{DIV}(33, 7)$

Example 6

Finding the remainder after dividing 33 by 7:

Result $\leftarrow \text{MOD}(33, 7)$

Example 7

Total $\leftarrow 1 + (2 * 3)$

Total $\leftarrow (1 + 2) * 3$

The first line will result in 7 ($3 * 2 = 6 + 1 = 7$). The second line will result in 9 ($1 + 2 = 3 * 3 = 9$).

Sequence

Sequence is the first of three constructs within programs. A sequence is a series of statements that are executed (run) once, in the order they are written.

Example 1

```
OUTPUT("Enter a colour")  
INPUT Colour  
OUTPUT("Enter your name")  
INPUT Name  
OUTPUT(Name , " your favourite colour is " , Colour)
```

This is a sequence. It has 3 lines are executed once, and in the order they are written (line 1, then 2 then 3).

Example 2

```
X←1  
Y←2  
Z←3  
Total ← X + Y + Z  
OUTPUT("Enter the first value")  
INPUT Value1
```

Checkpoint

1. Give the result from the following calculations:

- $10 + 20$
- $20 / 2$
- $100 * 4$
- $50 - 15$
- $20 \text{ DIV } 2$
- $39 \text{ DIV } 6$
- $20 \text{ MOD } 2$
- $40 \text{ MOD } 6$

2. Write a program to take a number as input, multiply it by 2 and output the result.

3. Write a program to store the numbers 10 and 12 in constants, add them together and then output the result.

4. Write a program to ask a user to enter their age and name, then output a message that uses both values, e.g. Hello Suzie you are 15 year old.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

[illegible]

Selection

Selection is the second of the three constructs. In selection a condition is checked and this determines which, if any, code is run. There are two forms of selection, IF statements and CASE statements.

Conditions need logical operators. These allow for comparisons to be made. Table describes these different operators. Each statement using a logical operator results in TRUE or FALSE.

Logical Operator	Description	Example
= or ==	Equals to	10 = 10? would give TRUE. 10 is equal to 10. 10 = 2? would give FALSE. 10 is not equal to 2.
<> or !=	Not equal to	10 <> 10? would give FALSE. 10 is not, not equal to 10. 10 <> 2? would give TRUE. 10 is not equal to 2.
<	Less than	10 < 11? would give TRUE. 10 is less than 11. 10 < 10? would give FALSE. 10 is not less than 10. 11 < 10? would give FALSE. 11 is not less than 10.
<=	Less than or equal to	10 <= 11? would give TRUE. 10 is less than or equal to 10. 10 <= 10? would give TRUE. 10 is less than or equal to 10. 11 <= 10? would give FALSE. 11 is not less than or equal to 10.

Logical Operator	Description	Example
>	Greater than	10 > 11? would give FALSE. 10 is not greater than 11. 10 > 10? would give FALSE. 10 is not greater than 10. 11 > 10? would give TRUE. 11 is greater than 10.
>=	Greater than or equal to	10 >= 11? would give FALSE. 10 is not greater than or equal to 11. 10 >= 10? would give TRUE. 10 is greater than or equal to 10. 11 >= 10? would give TRUE. 11 is greater than or equal to 10.

IF statements

The command IF is followed by a condition that is created using the logical operators. There are three stages of IF statements; IF, ELSE and ELSEIF.

IF has one comparison and the code inside the IF will only run if that condition is True. If it is not true, the code in the IF statement will not run.

It follows the structure:

```
IF comparison THEN
    Statements that run if the comparison is true
ENDIF
```

Example 1

This program will check the value in the variable num1 is equal to 10. If it is, it will output the word True.

```
Num1 ← 10
IF Num1 = 10 THEN
    OUTPUT("True")
ENDIF
```

Example 2

This program will check if the value input is greater than the one stored in the variable.

```
OUTPUT("Enter a number")
INPUT ValueInput
StoredValue ← 100
IF ValueInput > StoredValue THEN
    OUTPUT("It is more than 100")
ENDIF
```

ELSE

This is added within an IF statement. If the IF statement's condition is false then the ELSE will

run. You can only ever have one ELSE in an IF statement. ELSE follows the structure:

IF comparison THEN
Statements that run if the comparison is true

ELSE
 Statements that run if the comparison is false
ENDIF

Example 1

In this example if the two values are the same it outputs "That's correct". If they are not the same then the ELSE runs, it will output "Incorrect".

```
Num ← 10
OUTPUT("Enter a number")
INPUT Guess
IF Num = Guess THEN
    OUTPUT("That's correct")
ELSE
    OUTPUT("Incorrect")
ENDIF
```

Example 2

In this example, it will output the smallest number, or one of the numbers if they are both the same.

```
Value1 ← 10
Value2 ← 20
IF Value1 < Value2 THEN
    OUTPUT(Value1)
ELSE
    OUTPUT(Value2)
ENDIF
```

ELSEIF

This allows for a second condition to be used within the same IF statement. If the first condition is False, then a second condition can be checked.

ELSEIF follows the structure:

```
IF comparison1 THEN  
    Statements that run if the comparison is true  
ELSEIF comparison2 THEN  
    Statements that run if comparison1 is false, and comparison2 is true  
....as many ELSEIFs as you need  
ELSE  
    Statements that run if none of the comparisons are true  
ENDIF
```

Example 1

This will output which number is greater, or nothing will output if they are the same.

```
Num1 ← 10  
Num2 ← 20  
IF Num1 < Num2 THEN  
    OUTPUT(Num2)  
ELSEIF Num2 < Num1 THEN  
    OUTPUT(Num1)  
ENDIF
```

Example 2

This uses ELSEIF and an ELSE to output the largest number.

```
IF Num1 > Num2 THEN  
    OUTPUT(Num1)  
ELSEIF Num2 > Num1 THEN  
    OUTPUT(Num2)  
ELSE  
    OUTPUT("They are the same")  
ENDIF
```

Example 3

This uses multiple ELSEIFs.

```
IF Age < 14 THEN  
    OUTPUT("You are not old enough")  
ELSEIF Age < 16 THEN  
    OUTPUT("You need an adult present")  
ELSEIF
```

CASE statements

A SELECT CASE statement allows the program to take one variable or value, and then have lots of different options depending what it is equal to.

CASE follows the structure:

CASE OF variable value1:

Statements that run if CASE value1 is true value2:

Statements that run if CASE value1 is false, and value2 is true

OTHERWISE

Statements that run if none of the comparisons are true.

ENDCASE

A case can have as many CASE statements as needed, but can only ever have a maximum of one default (this runs if none of the comparisons are true).

Example 1

Using a SELECT CASE to output a grade for an in-lesson test. (No example is given for Python as it does not have a CASE construct and no example is given for Java as it does not support switch statements with comparisons, e.g. < or >.)

Score ← INPUT("Enter score")

CASE OF score:

>=80: OUTPUT ("A")

>=70: OUTPUT("B")

>=60: OUTPUT("C")

>=50: OUTPUT("D")

OTHERWISE OUTPUT("U")

ENDCASE

Example 2

Output a message depending on which number is entered. (No example is given for Python as it does not have a CASE construct.)

```
OUTPUT("Enter a number, 1 to 5")  
INPUT Choice  
CASE OF Choice:  
  1: OUTPUT("Menu option 1")  
  2: OUTPUT("Menu option 2")  
  3: OUTPUT("Menu option 3")  
  4: OUTPUT("Menu option 4")  
  5: OUTPUT("Menu option 5")  
  OTHERWISE OUTPUT("Invalid choice")  
ENDCASE
```

Boolean operators

There are three Boolean operators that you can use to join conditions: the AND operator, the NOT operator and the OR operator.

Boolean Operator	Description	Example
AND	If both conditions are true, the result is true. If one or both conditions are false, the result is false.	IF $1 = 1$ AND $2 = 2$ This will return TRUE. The left of the AND is true, and the right of the AND is true. IF $1 = 1$ AND $1 > 2$ This will return FALSE. The left of AND is true, but the right of AND is false. IF $1 < -2$ AND $0 < -1$ This will return FALSE. Both comparisons are false, so the result is false.
OR	If one, or both, conditions are true, the result is true. If both conditions are false, the result is false.	IF $1 = 1$ OR $2 = 2$ This will return TRUE. The left of the OR is true, and the right of the OR is true. IF $1 = 1$ OR $1 > 2$ This will return TRUE. The left of OR is true, but the right of OR is false. IF $1 < -2$ OR $0 < -1$ This will return FALSE. Both comparisons are false, so the result is false.

Boolean Operator	Description	Example
NOT	Reverse the condition. If the condition is True it becomes False.	IF NOT(1 = 1) The brackets equal to TRUE, 1 equals 1. The NOT makes it FALSE, so it becomes 1 does not equal 1. IF NOT (End of File) This is used with file handing. End of File will return TRUE if there is no data left in the file. The NOT turns this to false. So while not at the end of the file.

Example 1

This will output the first message if both test marks are greater than or equal to 90. If only one mark is greater than or equal to 90 then the second message will output.

```
OUTPUT("Enter the mark for test 1")  
INPUT Mark1  
OUTPUT("Enter the mark for test 2")  
INPUT Mark2  
IF Mark1 >= 90 AND Mark2 >= 90 THEN  
    OUTPUT("Well done you got top marks on both tests")  
ELSEIF Mark1 >= 90 OR Mark2 >= 90 THEN  
    OUTPUT("Well done you got top marks on one of the tests")  
ELSE  
    OUTPUT("You didn't quite get top marks on the tests, try  
    again next time")  
ENDIF
```

Example 2

Output the highest number out of three that are input:

```
OUTPUT("Enter 3 numbers")  
INPUT Number1  
INPUT Number2  
INPUT Number3  
IF Number1 > Number2 AND Number1 > Number3 THEN  
    OUTPUT(Number1)  
ELSEIF Number2 > Number3 THEN  
    OUTPUT(Number2)  
ELSE  
    OUTPUT(Number3)  
ENDIF
```

Zain Merchant

- [illegible]

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Iteration

An iteration or loop is a programming construct where statements are run either a finite number of times, until a condition is true or while a condition is true.

There are three types of loop: count-controlled, pre-condition and post-condition.

Count-controlled

This type of loop uses a counter to run a set number of times. The most common count-controlled loop is the for loop. This has the structure:

```
FOR variable ← start value TO endvalue  
    Code that runs repeatedly  
NEXT variable
```

The loop will run from the start value to the end value, increasing by 1 each time. If the start value is 1 and the end value is 10, it will run 10 times (1, 2, 3, 4, 5, 6, 7, 8, 9 and 10).

Example 1

Output the numbers 1 to 10:

```
FOR X ← 1 TO 10  
    OUTPUT(X)  
NEXT X
```

Example 2

Output the 12 times table from 1 to 12:

```
FOR Count ← 1 TO 12  
  OUTPUT(Count * 12)  
NEXT Count
```

Example 3

Add together the first 100 numbers:

```
Total ← 0  
FOR Number ← 1 TO 100  
  Total ← Total + Number  
NEXT Number
```

You can change the amount that you increase the variable by each time you loop. This is by using STEP. STEP 1 will increase the counter by 1 each time. STEP -1 will decrease the counter by 1 each time. STEP 0.5 will increase the counter by 0.5 each time.

Example 1

Output the numbers 10 to 1:

```
FOR Number ← 10 TO 1 STEP -1  
  OUTPUT(Number)  
NEXT Number
```

Example 2

Output the numbers from 11 to 20, increasing by 0.5 each time. (No example is given for Python as it does not support stepping in decimals.)

```
FOR Value ← 11 TO 20 STEP 0.5  
  OUTPUT(Value)  
NEXT Value
```


Pre-condition

A pre-condition loop tests the condition before starting the loop. This means that if the condition is false, the code inside the loop will not run. It loops while the condition is true. It stops looping when the condition is false.

A WHILE loop is a pre-condition loop. It has the structure:

```
WHILE condition DO  
    Code that will run when the condition is true  
ENDWHILE
```

Example 1

Looping while the user enters "Yes".

```
InputValue ← "Yes"  
WHILE InputValue = "Yes" DO  
    InputValue ← INPUT("Do you want to continue?")  
ENDWHILE
```

Example 2

Outputting the numbers 1 to 10:

```
Number ← 1  
WHILE Number < 11 DO  
    OUTPUT(Number)  
    Number ← Number + 1  
ENDWHILE
```

Example 3

Asking the user to enter a number until they guess the stored number correctly:

```
Number ← 5  
Guessed ← FALSE  
WHILE Guessed = FALSE DO  
    OUTPUT("Guess the number")  
    INPUT Guess  
    IF Guess = Number THEN  
        Guessed ← TRUE  
    ENDIF  
ENDWHILE
```

Post-condition loop

A post-condition loop runs the code inside the loop once, and then checks the condition at the end of the loop. This means that the code will always run once.

A REPEAT UNTIL loop is a post-condition loop. It has the structure:

REPEAT

Code that runs inside the loop

UNTIL Condition

In this case it continues until the Condition becomes True. It loops while the condition is False.

Example 1

Looping until the user enters Yes. (No example is given for Python as it does not have an in-built post-condition loop.)

```
REPEAT  
  OUTPUT("Do you want to stop?")  
  INPUT Answer  
UNTIL Answer = "Yes"
```

Example 2

Outputting the numbers 1 to 10. (No example is given for Python as it does not have an in-built

post-condition loop.)

```
Number ← 1  
REPEAT  
  OUTPUT(Number)  
  Number ← Number + 1  
UNTIL Number > 10
```

Example 3

Asking the user to enter a number until they guess the correct number.

```
NumberToGuess ← 15
```

```
REPEAT
```

```
    OUTPUT("Guess the number")
```

```
    INPUT Guess
```

```
UNTIL Guess = NumberToGuess
```

Zain Merchant

-
- This image shows a full page of primary-ruled paper. It features multiple horizontal rows, each defined by two parallel dashed lines. The lines are evenly spaced across the entire page, providing a guide for handwriting practice. There are no margins, text, or other markings present.

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Totalling

Totalling is adding together a set of values. To write a program to total you need to:

- Initialise the total to 0.
- Add the values together (either individually or within a loop).

Example 1

Asking the user to enter 10 numbers and totalling them:

```
Total ← 0  
FOR Counter ← 0 TO 10  
OUTPUT("Enter a number")  
Total ← Total + INPUT  
NEXT Counter  
OUTPUT("The total is " & Total)
```

Example 2

Total the data in an array of 100 elements:

```
Total ← 0  
FOR Count ← 0 TO 99  
Total ← Total + ArrayData[Count]  
NEXT Count  
OUTPUT(Total)
```

Counting

Counting is working out how many of something there are. For example how many numbers were entered that were over 10. To write a program to count you need to:

Initialise a counter variable to 0.

Increment (add one to) the counter each time an item is entered, or found.

Example 1

Count how many numbers the user enters until they say to stop:

```
Count ← 0
Continue ← "Yes"
WHILE Continue = "Yes" DO
    OUTPUT("Do you want to continue?")
    INPUT Continue
    Count ← Count + 1
ENDWHILE
OUTPUT("You continued " & Count - 1 & " times")
```

Example 2

Count how many numbers in an array of 100 elements are more than 50:

```
Count ← 0
FOR X ← 0 TO 99
    IF ArrayData[X] > 50 THEN
        Count ← Count + 1
    ENDIF
NEXT X
```


Zain Merchant

-
- This image shows a full page of a handwriting practice worksheet. It consists of multiple rows of horizontal dashed lines spaced evenly down the page, providing a guide for letter height and placement. The background is plain white, and there are no margins or additional markings.

[illegible]

String manipulation

A string is a piece of text. This could be made up of characters, numbers and/or symbols. There are lots of different string manipulators that you can use; these let you alter strings, find values in strings, etc. The two you need to know are length and substring.

Length

This command will return the number of characters in a string. It has the structure: `LENGTH(string)`.

Example 1

```
LENGTH("hi")
```

would return 2.

Example 2

```
LENGTH("0123")
```

would return 4.

Substring

This command will return some of the characters in the string. It has the structure:

`SUBSTRING(string, start character, number of characters)`. Depending on your language, the first character could be in position 0 or position 1.

Example 1

Using substring:

```
SUBSTRING("Hello", 0, 1)
```

This will start at character 0 and take 1 character. It will return "H".

Example 2

Using substring:

```
SUBSTRING("Goodbye", 4, 3).
```

This will start at character 4 and take 3 characters. It will return "bye".

Example 3

Output the length of a string that the user inputs:

```
InputString ← INPUT("Enter a string")
StringLength ← LENGTH(InputString)
OUTPUT(InputString & " is " & StringLength & " characters long")
```

Example 4

Output the first 4 characters in a string:

```
StringData ← "Goodbye"
NewMessage ← SUBSTRING(StringData, 0, 4)
OUTPUT(NewMessage)
```

Example 5

Output each letter of a string one character at a time. Depending on your language, the stopping condition might be the length, or the length -1 depending on whether the first character is 0 or 1.

```
OUTPUT("Enter a message")
INPUT StringInput
FOR Count ← 0 to LENGTH(StringInput) - 1
    Character ← SUBSTRING(StringInput, Count, 1)
    OUTPUT(Character)
NEXT Count
```

Example 6

Output the last 3 characters in a string:

```
OUTPUT("Enter a message")  
INPUT StringInput  
NewString ← SUBSTRING(StringInput, LENGTH(StringInput) - 3, 3)  
OUTPUT(NewString)
```

Upper and lower

The characters a–z can be converted into uppercase and the characters A–Z can be converted into lowercase. This can be done to an individual character, or to an entire string at the same time. If a character is already in upper case, trying to convert it to upper case will not change it.

UPPER(string) LOWER(string)

Example 1

Using UPPER with a string:

```
UPPER("Hello")
```

will return "HELLO"

Nested statements

A nested statement is one or more selection and/or iteration statements inside another selection/iteration statement. This could be an IF statement inside an IF statement or a loop inside an IF statement or an IF statement in a loop or a loop within a loop. You might have already used these without realising they were called nested statements.

The position of the start and end of these constructs are important. If, for example, a loop starts inside an IF statement, the loop must also finish inside the same IF statement.

Example 1

Count how many numbers entered are more than 10, and how many are equal to 10:

```
MoreThan10 ← 0
EqualTo10 ← 0
FOR X ← 0 TO 99
    OUTPUT("Enter a number")
    INPUT Number
    IF Number > 10 THEN
        MoreThan10 ← MoreThan10 + 1
    ELSEIF Number = 10 THEN
        EqualTo10 ← EqualTo10 + 1
    ENDIF
NEXT X
```

This code has an IF statement nested inside a count-controlled loop.

Example 2

Loop counting how many values in an array are more than or equal to 100, and then stop counting:

```

Number ← 0
Count ← 0
WHILE Number < 10 DO
  DataInArray ← ArrayData[Count]
  Count ← Count + 1
  IF DataInArray >= 100 THEN
    Number ← Number + 1
  ENDIF
ENDWHILE

```

This has an IF statement inside a pre-condition loop.

Example 3

Output only the vowels in a message input if user selects option 1:

```

OUTPUT("Enter 1 or 2")
INPUT Choice
IF Choice = 1 THEN
  OUTPUT("Enter a word")
  INPUT Word
  FOR Count ← 0 to LENGTH(Word)-1
    Character ← SUBSTRING(Word, Count, 1)
    IF Character = "a" OR Character = "e" OR Character = "i" OR Character = "o"
      OR Character = "u" THEN
      OUTPUT(Character)
    ENDIF
  NEXT Count
ENDIF

```

This has a FOR loop inside an IF, and another IF inside the FOR loop.

Zain Merchant

- [illegible]

[illegible]

Subroutines

A subroutine is a self-contained piece of code that has an identifier (name), and it can be called from anywhere in the main program.

When you decompose a problem into sub-systems, each of the sub-systems can be written as an individual subroutine. You can then call that subroutine when you need to use it.

Subroutines are useful because they reduce code. You write the subroutine once, and then you can call it as many times as you need to, instead of having to re-write it every time. Each time you re-write it there is a chance of an error, so this reduces the chances of this error.

There are two types of subroutine: procedures and functions. A function returns a value to the program that called it. A procedure does not return a value.

Procedures and functions can both take one or more values as parameters. These are values that are sent to the subroutine. Parameters will be introduced after the basics of procedures and functions.

Procedures

A procedure runs the code inside it, and does not return a value. The structure of a procedure is:

```
PROCEDURE identifier()  
    code to run inside the function  
ENDPROCEDURE
```

The identifier is then used in the main program.

Example 1

A procedure to output the numbers 1 to 10:

```
PROCEDURE Output1To10()  
    FOR Count ← 1 to 10  
        OUTPUT(Count)  
    NEXT Count  
ENDPROCEDURE
```

The main program can then call the procedure with the code:

```
Output1To10()
```

Example 2

A procedure to take two numbers from the user and multiply them together:

```
PROCEDURE Multiply()  
    OUTPUT("Enter a number")  
    INPUT Num1  
    OUTPUT("Enter a second number")  
    INPUT Num2  
    Total ← Num1 * Num2  
ENDPROCEDURE
```

The procedure can be called in the main program with the code:

```
multiply()
```

Function

A function returns a value to the program that called it. This can be by either using the RETURN command, or saving a value to the function's identifier. Once a value is returned, the function stops running, so it cannot have any code after otherwise this will not run.

It has the structure:

```
FUNCTION identifier()  
    Code to run in the function  
    RETURN value  
ENDFUNCTION
```

When the function is called it returns a value, so something needs to happen with this value. It could be output, e.g.

```
OUTPUT(function identifier)
```

or it could be saved in a variable, e.g.

```
variable identifier = function identifier
```

Example 1

Write a function to ask the user to enter two values, add them together and return the value:

```
FUNCTION Multiply()  
    OUTPUT("Enter a number")  
    INPUT Num1  
    OUTPUT("Enter another number")  
    INPUT Num2  
    RETURN Num1 * Num2  
ENDFUNCTION
```

To output the return value the main program can use:

```
OUTPUT(Multiply())
```

Example 2

Write a function to total all the values in an array with 50 elements and then return the total:

```
FUNCTION TotalValues()  
  Total  $\leftarrow$  0  
  FOR X  $\leftarrow$  0 TO 49  
    Total  $\leftarrow$  Total + Array[X]  
  NEXT X  
  RETURN Total  
ENDFUNCTION
```

To store the return value in a variable in the main program:

```
Total = TotalValues()
```

Scope

The scope of a variable is the areas within a program that it can be accessed. There are two scopes: global and local.

If you declare a variable (or constant, or array) as global then it means it can be accessed by any part of the program. That includes the main program and any subroutines. In most languages this means declaring it at the top of the program.

Example 1

Declaring a global variable, then outputting its value twice. Once in the main program, and once in a procedure call.

```
GLOBAL Data
PROCEDURE OutputData()
  OUTPUT(Data)
ENDPROCEDURE
//main program
Data ← 1
OUTPUT(Data)
OutputData()
```

If you declare a variable (or constant, or array) as local, then it can only be accessed in the part of the code where it is declared. If you declare it first in a subroutine, then it can only be accessed within that subroutine. If you declare it in the main program, it can only be accessed in the main program.

Example 2

Creating a local variable to the main program and outputting it twice. Once in the main program, and once from a subroutine where it is sent as a parameter.

```
PROCEDURE OutputData(DataParameter)
  OUTPUT(DataParameter)
ENDPROCEDURE
Data ← 1
OUTPUT(DataParameter)
OutputData(Data)
```

Parameters

A parameter is a value that is sent from the main program to the subroutine (procedure or function). Parameters are declared inside the brackets after the subroutines name, e.g.

```
PROCEDURE identifier(parameter1, parameter2 ...) ENDPROCEDURE
```

or

```
FUNCTION identifier(parameter1, parameter2 ...) ENDFUNCTION
```

If a subroutine is declared with parameters, then it must be called with the same number of parameters. For example:

```
PROCEDURE Total(Num1, Num2)
ENDPROCEDURE
```

This has two parameters. When the procedure is called it must have 2 numbers sent to it. This could be numbers, e.g.

```
Total(1,2)
```

or variables. e.g.

```
Total(Number1, Number2)
```

Example 1

A function takes two numbers, divides them and returns the result:

```
FUNCTION Division(First, Second)
  RETURN First / Second
ENDFUNCTION
```

The main program sends 10 and 2, then outputs the return value.

```
OUTPUT(Division(10, 2))
```

Checkpoint

1. What is the difference between a function and a procedure?

2. Consider the following function:

```
FUNCTION Calculate(Num1, Num2)
    Num1 ← Num1 * 2
    Num2 ← Num2 + Num1
    RETURN(Num1 + Num2)
ENDFUNCTION
```

3. What will the following statement output?

```
OUTPUT(Calculate(1,2))
```

4. Write a program statement to call the following function with the parameter 100 and output the return value.

```
FUNCTION FindValue(Number)
    Number ← Number + INPUT
    RETURN Number
ENDFUNCTION
```

5. Write a procedure to take three numbers as parameters and output the largest.

6. Write a function that takes two strings as parameters. It takes the first 3 characters of each string and combines them, returning the resulting string.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

[illegible]

Library routines

A program library is a set of subroutines that are pre-written and that can be called within a program.

In some programming languages the operators for MOD and DIV are library functions. In other programming languages they are just operators. For example, 2 MOD 4 is the same as MOD(2, 4).

Two other library routines that you need to know are ROUND and RANDOM.

ROUND

This will take a real number (decimal) and limit how many numbers there are after the decimal point.

For example ROUND(10.123, 1) will take the number 10.123 and only leave 1 number after the decimal point, returning 10.1.

ROUND(4.8293, 2)

will return 4.82.

As with functions, the values it returns need to be used. This could be done by outputting the return value, or saving it in a variable, e.g.

RoundedValue ← ROUND(77.293, 1)

RANDOM

This will generate a random number between two values that it takes as parameters. For example,

```
RANDOM(10, 20)
```

will return a number between 10 and 20.

```
RANDOM(1, 4)
```

will return a number between 1 and 4.

As with functions, the values it returns and therefore need to be used. This could be by outputting

the return value, or saving it in a variable, e.g.

```
randomNumber = RANDOM(1, 100)
```

Maintainable programs

When you write a program there are several things to take into consideration to make it a maintainable program. This is so that when you come back to it in a week, or a year, you can still understand what all of the code does. It might be you are writing a program that someone else needs to understand, so you need to make it understandable to someone who does not know what the program does.

Meaningful identifiers

Variables, constants, subroutines and arrays all have identifiers (names). If you call a variable X, then there is no indication of what it is storing or what its purpose is. If instead, it is called Total, then you know that it is storing a total.

The identifiers for subroutines are usually descriptions of their function. For example, a procedure to output the numbers 1 to 10 could be called Function1, but then there is no indication of what it does. Instead, it could be called Output1To10.

Comments

A comment is a description of a line of code, or section of code. To write a comment you use a special character or characters, for example, //. This tells the program not to execute the text after this symbol.

You do not need to comment every line of code, for example, the statement `Count = 0` does not need commenting, it is clear that it is storing 0 in the variable count.

Example 1

The function of the FOR loop is written as a comment:

```
FOR Count ← 0 TO 9 //Output the first 10 elements in the array
  OUTPUT(Array[Count])
NEXT Count
```

Example 2

```
OUTPUT("Enter a number")
INPUT Num1
OUTPUT("Enter a number")
INPUT Num2
//find and output the largest number
IF Num1 > Num2 THEN
    OUTPUT(Num1)
ELSE
    OUTPUT(Num2)
ENDIF
```

Zain Merchant

-
- This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

This image shows a single page of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Arrays

An array is a data structure. It allows you store multiple pieces of data in one structure with one identifier. In an array, each data item must be of the same data type. If it stores integers, then all values must be integers. If it stores strings, then all values must be strings.

1-dimensional arrays

A 1-dimensional array has just one row of data. The best way to visualise an array is with a table:

Index	0	1	2	3	4
Data	10	5	90	26	87

This array has 5 spaces. Each space has an index. In this array the first data item is in position 0, the data value is 10. In the second array space (index 1), the number 5 is stored.

Arrays can be 0-indexed or 1-indexed. This depends on the programming language that you use. Some arrays start with 0 as the first space. Some arrays start with 1 as the first space.

Arrays use brackets after the identifier to indicate the index you want to access. For example, `Array[0]` is accessing the first element in the array named `Array`. `MyData[3]` is accessing the fourth element in the array named `MyData`.

Putting data into an array

You need to know the array identifier and the position where you want to store the data.

Example 1

Store the colour "red" in the first position of the array named `Colour`:

```
Colour[0] ← "red"
```


Example 2

Store the colour "yellow" in the second position of the array named Colour:

```
Colour[1] ← "yellow"
```

Example 3

Store the number 20 in the sixth position of the array named Numbers:

```
Numbers[5] ← 20
```

Getting data out of an array

To access data in an array you need to know the identifier and the position of the data you want. This will be a value, so you need to do something with this value, e.g. store it in a variable.

Example 1

Output the first value in the array Colours:

```
OUTPUT(Colours[0])
```

Example 2

Store the second value in the array Colours in a variable:

```
TheColour ← Colours[1]
```

Example 3

Add 10 to the third value in the array Numbers:

```
Value ← 10 + Numbers[2]
```

Using variables as indices

The index in the array might be a variable that stores a number.

Example

Ask the user which array element to output from the array colours:

```
OUTPUT("Enter the array element you want to output")  
INPUT ToOutput  
OUTPUT(Colours[ToOutput])
```

Using iteration to read and write

If you have a set of values in an array you can use iteration to loop through each of the elements in turn. For example, you might want to output all of the values one at a time. You could add together all of the values in an array and output the total. You might want to take 10 values in from the user and store each one in the array.

These are all best done using a count-controlled loop. This is because you usually know how many values you want to enter, or how many values there are in the array that you want to work through.

Example 1

Output all 10 elements in the array Colours:

```
FOR Count ← 0 TO 9  
  OUTPUT(Colours[Count])  
NEXT Count
```

Example 2

Ask the user to input 20 numbers and store each in the array Numbers:

```
FOR counter ← 0 TO 19  
  OUTPUT("Enter a number")  
  INPUT Numbers[Counter]  
NEXT counter
```

Example 3

Searching the values in the array values that has 50 values, for the data input by the user:

```
ValueToFind ← INPUT("Enter the value to find")  
FOR Count ← 0 TO 49  
  IF Values[Counter] = ValueToFind THEN  
    OUTPUT("Found it")  
  ENDIF  
NEXT Count
```

2-dimensional arrays

A 2-dimensional array is best viewed as a table with rows and columns.

index	0	1	2	3	4
0	10	5	90	26	87
1	3	15	74	62	5
2	7	10	85	4	24

In a 2-dimensional array there are two indices. For example, from the table:

Position[0, 0] is 10.

Position[0, 2] is 7.

Position[4, 2] is 24.

Putting data into an array

You need to know which position, i.e. both indices, the across and the down.

Example 1

Store "red" in the first position in the array Colours:

```
Colour[0, 0] ← "red"
```

Example 2

Store 10 in the array Data, in element 4 across and 3 down:

```
Data[4, 3] ← 10
```

Getting data out of an array

You need to know both indices to access the data.

Example 1

Output the data in the array ArrayData, element 5 across and 1 down:

OUTPUT(ArrayData[4, 1])

Example 2

Access the data in the array colours, in the first element across and the third down:

ColourAccessed ← Colours[0, 2]

Using variables as indices

Each index can be stored in a variable in the same way as they can be in a 1D array.

Example 1

Output the data in element 4,3:

First ← 4

Second ← 3

OUTPUT(ArrayData[First, Second])

Example 2

Ask the user which element to store in **data**:

OUTPUT("Enter dimension 1")

INPUT Index1

OUTPUT("Enter dimension 2")

INPUT Index2

Data ← Array[Index1, Index2]

Using iteration to read and write

Due to the two dimensions, you need two nested loops to read through all the data elements. If you think about the table again, one loop goes through the columns and one loop goes through the rows.

The first loop will check row 1. The column will change from 0, 1, 2, 3, 4. The row will stay the same at 0.

index	0	1	2	3	4
0	10	5	90	26	87
1	3	15	74	62	5
2	7	10	85	4	24

The first loop will check row 2. The column will change from 0, 1, 2, 3, 4. The row will stay the same at 1.

It is best to use count controlled loops to go through the array. It has the structure:

FOR row ← first index to last index

FOR column ← first index to last index

Code to run

NEXT row

NEXT count

Example 1

DataArray has 10 elements by 3 elements. Output all of the elements in the array:

FOR Row ← 0 TO 2

FOR Column ← 0 TO 9

OUTPUT(DataArray[Column, Row])

NEXT Row

NEXT Count

Example 2

Search a 2-dimensional array, with 50 elements by 100 elements, for the value input by the user:

```

OUTPUT("Enter a number to search for")
INPUT SearchValue
FOR Row ← 0 TO 49
  FOR Column ← 0 TO 99
    IF DataArray[Row, Column] = SearchValue THEN
      OUTPUT("Found it at " & Column & " " & Row)
    ENDIF
  NEXT Column
NEXT Row

```

Example 3

Find and output the total of all elements in the each of the first dimensions, in an array of 10 elements by 15 elements:

```

FOR Row ← 0 TO 9
  Total ← 0
  FOR Column ← 0 TO 14
    Total ← Total + TheArray[Row, Column]
  NEXT Column
  OUTPUT("Index " & Row & " has the total " & Total)
NEXT Row

```

Example 2

A procedure takes 2 values and outputs all the numbers between the first number to the second:

```
PROCEDURE OutputNumbers(Num1, Num2)  
  FOR Count ← Num1 TO Num2  
    OUTPUT Count  
  NEXT Count  
ENDPROCEDURE
```

The main program taking two values from the user.

```
OUTPUT("Enter the smallest number")  
INPUT FirstNumber  
OUTPUT("Enter the largest number")  
INPUT SecondNumber  
OutputNumbers(FirstNumber, SecondNumber)
```


Zain Merchant

2. Explain why the following code will result in an error.

MyData[1] ← 4

MyData[3] ← “9”

- A 2-dimensional array, MyData, has 20 elements by 5 elements. Write a function that takes a parameter search value. The function should search MyData and return either TRUE if the parameters is in MyData, or FALSE if the parameters is not in MyData.

This image shows a full page of a handwriting practice worksheet. It consists of multiple horizontal rows, each defined by two parallel dashed lines. The rows are evenly spaced and extend across the entire width of the page, providing a guide for letter height and placement. There is no text or other markings on the page.

[illegible]

File handling

If you have data in a program, when you stop or close a program all of that data is lost. If you need to save data to use again later then you need to save it externally into a text file. The storage and access of data in a file is called file handling.

Reading from a file

You need to be able to read a single item of data, or a single line of text. This means all of the data will be on the first line in the text file, so you do not need to check how many lines of text are in the file. Once you have read in the data you can then manipulate it, for example, if it is a line of text you can split it into individual words, or use it as one item of data.

To read a value from a file you need to:

- Open the file using its filename (the filename will be a string value).
- Read the data value, and do something with it (e.g. output it, store it in a variable).
- Close the file.

You can use the pseudocode commands:

```
OPEN filename  
variable identifier ← READ (filename)  
CLOSE filename
```

Example 1

Reading and outputting a word stored in the text file data.txt:

```
OPEN "data.txt"  
OUTPUT(READ("data.txt"))  
CLOSE "data.txt"
```

Example 2

Read and output the number stored in the file myNumber.txt by storing the filename in a variable:

```
Filename ← "myNumber.txt"  
OPEN Filename  
TheFileData ← READ(Filename)  
CLOSE(Filename)  
OUTPUT(TheFileData)
```

Writing to a file

The specification states that you will only need to write a single item of data or a line of text. This means you will be overwriting any data that is already in the file, you do not need to worry about appending (to add onto the end) to data that already exists, or writing multiple values to the same file.

To write a value to a file you need to:

- Open the file using its filename (the filename will be a string value).
- Write the data to the file.
- Close the file.

You can use the pseudocode commands:

```
OPEN filename  
WRITE data  
CLOSE filename
```

Example 1

Write the word "red" to the file colour.txt:

```
OPEN "colour.txt"  
WRITE "red"  
CLOSE "colour.txt"
```

Example 2

Write the number 100 to the file myData.txt storing the filename in a variable:

```
Filename ← "myData.txt"  
OPEN Filename  
WRITE 100  
CLOSE Filename
```

Checkpoint

1. Why do some programs need to store data in files?
2. What are the three stages that need to be followed to write data to a file?
3. Write a program to read in a value from the file `dataStore.txt` and output it.
4. Write a program to ask the user to input a filename, then store the word "house" in the file.

[illegible]

[illegible]

Common algorithms

A search algorithm checks a set of data to identify whether a specific value exists in the data, or not. One example of this is a linear search.

A sorting algorithm takes a set of data and rearranges it to be in a specific order, e.g. in ascending alphabetical order. One example of this is a bubble sort.

Linear search

In a program you might need to look for a specific value in a set of data. A linear search will check each item one at a time, starting with the first item and continuing until it either finds the item, or it checks the last value.

Example 1

Search this set of data for the number 4.

5	3	9	4	2	1	8
---	---	---	---	---	---	---

Compare the first number (5) with the search value (4). They are not the same.

Compare the second number (3) with 4. They are not the same.

Compare the third number (9) with 4. They are not the same.

Compare the fourth number (4) with 4. They are the same. It has been found.

Example 2

Search this set of data for the character "#".

"a"	"v"	"!"
-----	-----	-----

Compare the "a" with "#". They are not the same. Compare the "v" with "#". They are not the same. Compare the "!" with "#". They are not the same. There are no more values, so "#" is not found.

Example 3

The array dataArray stores a set of data under one identifier. Each element in the array has an index.

Index	0	1	2	3
Data	1	5	6	8

Search the array for the number 6

Is **DataArray[0] = 6**? *No*, **DataArray[0] = 1**

Is **DataArray[1] = 6**? *No*, **DataArray[1] = 5**

Is **DataArray[2] = 6**? *Yes*, **DataArray[2] = 6**

6 was found in index 2.

This can now be written as an algorithm:

```

FOR ArrayIndex = 0 to LENGTH(Array) -1 //loop through each
element
  //check if the current index is the data searched for
  IF DataArray[ArrayIndex] = SearchValue THEN
    //if it is, output the index where it is found
    OUTPUT "Found at " & ArrayIndex
  ELSE
    //if it not found, increment arrayIndex to check the
    next value
    ArrayIndex ← ArrayIndex + 1
  ENDIF
NEXT ArrayIndex

```

This algorithm is inefficient because if it finds the search value then it still continues searching. This is useful if you want to know whether the search value appears more than once.

You can make the algorithm more efficient by stopping as soon as the value is found, for example:

```

Found ← FALSE
ArrayIndex ← 0
//run while the value is not found, and you have not checked
//all elements
WHILE Found = FALSE AND ArrayIndex < LENGTH(Array) DO
  //check if the current index is the data searched for
  IF DataArray[ArrayIndex] = SearchValue THEN
    //if it is output where it was found
    OUTPUT "Found at " & ArrayIndex
    //set found to be true to stop the while loop running
    //again
    Found ← TRUE
  ELSE
    //if it is not found, increment ArrayIndex to the next
    value
    ArrayIndex ← ArrayIndex + 1
  ENDIF
ENDWHILE

```

Bubble sort

When writing a program you may need to sort a set of data into a specific order, for example, ascending numerical order or descending alphabetical order.

One method of sorting data is using a bubble sort.

A bubble sort takes the first 2 values; value 1 and 2, and compares them. If they are the wrong way around it swaps them. It then puts these back and takes values 2 and 3 and compares them. If they are the wrong way around it swaps them. This repeats until it has worked all the way through the list once. It then starts again with the first two values.

It is called a bubble sort because it acts like a bubble moving across all the elements to the end, then starting from the beginning.

There are two ways to tell the algorithm when to stop.

1. The algorithm has been through the entire list, the number of elements in the list $- 1$. So if there are 10 elements, the algorithm runs 9 times. If there are 100 elements, the algorithm runs 99 times. If there are X Elements, the algorithm runs $X \text{ Elements} - 1$ times. This is the easiest way to program it, but it is not efficient. All elements might be in the correct order to begin with, but it is still going to run through the same number of times.
2. Either the algorithm has run through the list $X \text{ Elements} - 1$ times, or it has run through the list, checking all of the elements, and it has not made any changes.

A bubble sort can be written as a pseudocode algorithm. There are different versions of the algorithm. Here are two different versions.

Version 1

Stopping only when it has run array length -1 times.

```
//loop array length - 1 number of times  
FOR NumberPasses  $\leftarrow$  0 to LENGTH(DataArray) - 1  
//loop through each element in the array  
  FOR Index  $\leftarrow$  0 to LENGTH(DataArray) - 1  
    //check if the data is in the correct order  
    IF DataArray[Index] > DataArray[Index + 1] THEN  
      //if not swap the items  
      Temp  $\leftarrow$  DataArray[Index]  
      DataArray[Index]  $\leftarrow$  DataArray[Index + 1]  
      DataArray[Index + 1]  $\leftarrow$  Temp  
    ENDIF  
  NEXT Index  
NEXT NumberPasses
```

Version 2

Stopping when there are no changes or when it has run array length – 1 times.

```
NumberPasses ← 0
//continue until one pass has no changes (changes = false)
//or it has looped array length - 1 times
WHILE Changes = FALSE or NumberPasses ≤ LENGTH(DataArray) - 1
DO
    //reset changes each time a new pass starts
    Changes ← FALSE
    //loop through each element in the array
    //check if the data is in the correct order
    FOR Index ← 0 to LENGTH(DataArray) - 1
        IF DataArray[Index] > DataArray[Index + 1] THEN
            //if not swap the items
            Temp ← DataArray[Index]
            DataArray[Index] ← DataArray[Index + 1]
            DataArray[Index + 1] ← Temp
            Changes ← TRUE
        ENDIF
    NEXT Index
ENDWHILE
```

Zain Merchant

- [illegible]

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Totalling

Totalling is adding a group of values together to give a total.

For example, a person buys items at a supermarket. The items cost: \$0.50, \$1.00, \$1.00, \$15.50, \$21.30 The total is $0.5 + 1 + 1 + 15.5 + 21.3 = \39.30 To do this in a program you need to:

Initialise a variable to store the total, e.g. $\text{Total} \leftarrow 0$.

Add the value to the current contents of the total, e.g. $\text{Total} \leftarrow \text{Total} + \text{Value}$.

Example 1

Totalling the 10 values input by the user:

```
Total ← 0 //Initialise the total variable to 0
FOR X ← 0 TO 9
  Total ← Total + INPUT("Enter a value") //Add the input to the total
NEXT X
OUTPUT Total
```

Example 2

Totalling the values in an array named dataArray:

```
TotalValue ← 0 //Initialise the total variable to 0
FOR Count ← 0 TO LENGTH(DataArray) - 1
  TotalValue ← TotalValue + DataArray[Count] //Add the input to the total
NEXT Count
OUTPUT Total
```


Counting

Counting is working out how many of something there are. For example, counting how many fingers you have or counting how many items you bought when you went shopping.

To do this in a program you need to:

Initialise a variable to start the count, e.g. **CountItems** $\leftarrow$ 0.

Add 1 to the count variable each time, e.g. **Count** $\leftarrow$ **Count** + 1.

Example 1

Count the number of items the user enters:

```
Count  $\leftarrow$  0 //initialise the count variable
Number  $\leftarrow$  0
WHILE Number  $\neq$  -1 DO // loop until the user enters -1
    Number  $\leftarrow$  INPUT("Enter a number. Enter -1 to stop")
    Count  $\leftarrow$  Count + 1 //add 1 to count each time a number is input
ENDWHILE
```

Example 2

Count how many numbers in an array are below 10, and how many are more than 20:

```
CountBelow10  $\leftarrow$  0 //initialise both counters to 0
CountAbove20  $\leftarrow$  0
FOR X  $\leftarrow$  0 to LENGTH(NumberArray) //loop through each array element
    //if the number is less than 10
    IF NumberArray[x] < 10 THEN
        CountBelow10  $\leftarrow$  CountBelow10 + 1 //increment the counter
    //if the number is greater than 20
    ELSEIF NumberArray[x] > 20 THEN
        CountAbove20  $\leftarrow$  CountAbove20 + 1 //increment the counter
    ENDIF
NEXT X
```

Finding the minimum

The minimum value is the smallest value within a set. For example, the marks for a class are entered and the lowest mark is the minimum.

To do this in a program you need to:

initialise a minimum variable to be a large value, beyond those that will be entered, e.g. **Minimum** $\leftarrow$ 9999.

Compare each value to the minimum variable, e.g. **IF Number** < **Minimum**.

If it is, it replaces the minimum value, e.g. **minimum** $\leftarrow$ **number**. This means the new value is the minimum, so the next time a value is compared, it is checking it with the new minimum value.

Example 1

Find the smallest number input by a user:

```
Minimum  $\leftarrow$  9999 //initialise minimum to large value
Number  $\leftarrow$  1
WHILE Number >= 1 DO //loop until the user enters 0
    Number  $\leftarrow$  INPUT("Enter a number or 0 to stop")
    IF Number < Minimum THEN // check if the number entered is smaller than //
        the current minimum
        Minimum  $\leftarrow$  Number //if true then make minimum because the number
    ENDIF
ENDWHILE
```

Example 2

Find the smallest number in an array of data:

```
Minimum ← 999999
FOR X ← 0 TO LENGTH(NumberArray)
  IF NumberArray[X] < minimum THEN
    Minimum ← NumberArray[X]
  ENDIF
NEXT X
```

Finding the maximum

The maximum value is the largest value within a set. For example, the marks for a class are entered and the highest mark is the maximum.

To do this in a program you need to:

- Initialise a maximum variable to be a small value, beyond those that will be entered, e.g. **Maximum** ← -9999.
- Compare each value to the maximum variable, e.g. **IF Number** > **Maximum**.
- If it is, it replaces the maximum value, e.g. **Maximum** ← **Number**. This means the new value is the maximum, so the next time a value is compared, it is checking it with the new maximum value.

Example 1

Find the largest number input by a user:

```

Maximum ← -9999 //initialise maximum to small value
Number ← 1
WHILE Number >= 1 DO //loop until the user enters 0
    Number = INPUT("Enter a number or 0 to stop")
    IF Number > Maximum THEN // check if the number entered is
        larger than //the current maximum
        Maximum ← Number //if true then make maximum because
        the number
    ENDIF
ENDWHILE

```

Example 2

Find the largest number in an array of data:

```

Maximum ← -1 // initialise the maximum value to a small number
FOR X ← 0 TO LENGTH(NumberArray)
    //check if the number entered is larger than the current maximum
    IF NumberArray[X] > Maximum THEN
        Maximum ← NumberArray[X] //if true store the value in maximum
    ENDIF
NEXT X

```

Finding the average

The average here is referring to the mean. This is the total of all the values added together, then divided by how many numbers there are.

For example, if the data is 1, 3, 5, 8, 4, 2, 6, 9 ,

the average = $(1 + 3 + 5 + 8 + 4 + 4 + 6 + 9) / 8 = 40 / 8 = 5$

To do this in a program you need to use the count and total from earlier in this section. The average is then calculated with the formulae **Total / Count**.

Example 1

Find the average of the numbers input by a user:

```
Count ← 0 //initialise the count to 0
Total ← 0 //initialise the total to 0
Number ← 1
WHILE Number >= 1 DO //loop until the user enters 0
    Number ← INPUT("Enter a number or 0 to stop")
    Total ← Total + Number //add the number entered to the total
    Count ← Count + 1 //add 1 to the count
ENDWHILE
```

Example 2

Find the average of the numbers in an array of data:

```
Count ← 0 //initialise the count to 0
Total ← 0 //initialise the total to 0
FOR X ← 0 TO LENGTH(NumberArray)
    Total ← Total + NumberArray[X] //add value to total
    Count ← Count + 1 //add 1 to count
NEXT X
```

Zain Merchant

- [illegible]

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Validation on input

Validation is the checking of data that is input to make sure it is reasonable, and/or within set bounds. For example, making sure that a number is entered for an age or limiting the range of numbers that can be entered.

There are different types of validation.

Range check

A range check assesses whether data is within one or two bounds. For example, an age must be between 0 and 100. A date must be before today's date.

Range checks can be programmed using selection to produce an error, or within a loop that keeps asking you enter a value until it is valid.

Example 1

Using selection:

This algorithm will check whether a number is higher than 1 and less than 10.

```
Number ← INPUT("Enter a number between 1 and 10")  
IF Number < 1 OR Number > 10 THEN  
    OUTPUT("Invalid")  
ELSE  
    OUTPUT("Valid")  
ENDIF
```

Example 2

Using a loop:

This algorithm will check if a number is higher than 1 and less than 10. It will continually ask for a value to be input until the number is valid.

```
Number ← INPUT("Enter a number between 1 and 10")
WHILE Number < 1 OR Number > 10 DO
  OUTPUT("Invalid please try again")
  Number ← INPUT()
ENDWHILE
```

Length check

A length check will check the number of characters that are present. This could in a string, for example, the length of "hello world" is 11, the space is also a character. It could be in a variable, for example, in this example, 8 will be output:

```
TheData ← "123 ABC!"
OUTPUT(LENGTH(TheData))
```

The length of a piece of data can be found using a variety of commands depending on the language you are using. The following are all valid and there are many more:

```
LENGTH(theData)
```

A length check can be programmed using selection or iteration.

Example 1

Using selection:

This algorithm will check if the data input has 10 or less characters to be invalid, otherwise (10 or more) it is valid.

```
Data ← INPUT()  
IF LENGTH(Data) < 10 THEN  
    OUTPUT("Invalid")  
ELSE  
    OUTPUT("Valid")  
ENDIF
```

Example 2

Using iteration:

This algorithm will also check the length but will continually ask for this to be input until it is 10 or more characters long.

```
Data ← Input()  
WHILE LENGTH(Data) < 10 DO  
    OUTPUT("Invalid, please try again")  
    Data ← INPUT()  
ENDWHILE
```

Type check

Data can be in different forms, including an integer (whole number), real (decimal number), string (any characters), Boolean (true or false). There are two ways of writing a type check:

1. Use a function such as `.GetDataType()` to return the data type, that you can then compare to the one you want.
2. Use a function such as `.IsInteger()` to return True if it is an integer and False otherwise.

Example 1

Using selection:

This algorithm will check if the data entered is an integer value to be valid.

INPUT Data

//check if the data entered is an Integer

IF Data.GetDataType() <> Integer THEN //if it is not an integer

OUTPUT ("Invalid")

ENDIF

Example 2

Using iteration:

This algorithm will continually ask for the data to be input until it is a string value.

INPUT Data

//loop while the data entered is not a string

WHILE Data.IsString = FALSE DO

OUTPUT("Invalid please try again")

INPUT Data

ENDWHILE

Presence check

A presence check makes sure that some data has been entered. Programming languages can make use of the value null, or for a string value an empty string represented by "".

Example 1

Using selection:

This algorithm outputs Invalid if there is no data entered.

```
INPUT Data  
IF Data = NULL THEN //check if the data entered is null  
  OUTPUT("Invalid")  
ENDIF
```

Example 2

Using iteration:

This algorithm continually takes an input while there is no data entered.

```
INPUT Data  
WHILE Data = "" DO //loop while there is nothing in data  
  OUTPUT("Invalid please try again")  
  INPUT Data  
ENDWHILE
```

Format check

Some data may need to be entered in a specific way, for example, a date must be: __/__/____, where each space is a number – a format check makes sure the data is entered this way. An ID number may need to be 1 number followed by 3 characters.

Example 1

Using selection:

An ID number entered needs to be 1 number followed by 2 characters. This algorithm checks the characters and outputs whether it meets the required format.

```
INPUT IdNumber  
//check if the first character is a number, and characters 2  
//and 3 are strings  
IF(SUBSTRING(IdNumber, 0, 1).IsNumeric = TRUE AND  
    SUBSTRING(IsNumber, 1, 2) = TRUE) THEN  
    OUTPUT("Valid")  
ELSE  
    OUTPUT("Invalid")  
ENDIF
```

Note that SUBSTRING(String, X, Y) starts at character x in the string, and returns y number of characters.

Example 2

Using a loop:

This algorithm checks whether an input is in the format two numbers, /, two numbers, /, four numbers.

```
INPUT Date  
//loop while the data is not in the correct format  
WHILE (SUBSTRING(Date, 0, 2).IsNumeric = FALSE OR //2 numbers  
    SUBSTRING(Date, 2, 1) <> "/" OR // 1st slash  
    SUBSTRING(Date, 3, 2).IsNumeric = FALSE OR //2 numbers  
    SUBSTRING(Date, 5, 1) <> "/" OR //2nd slash  
    SUBSTRING(Date, 6, 4).IsNumeric = FALSE) DO // 4 numbers, year  
    OUTPUT("Invalid")  
    INPUT Date  
ENDWHILE  
OUTPUT("Valid")
```

Check digit

A check digit is calculated from a set of numbers, and is input with the numbers. When the data has been input, the calculation is performed on the data again and the result is compared with the final number (the check digit).

Example

This algorithm calculates the check digit using the method 5 digit number input. 9999, 4 digits used to calculate 5th check digit, and compares it to the input value.

```

INPUT Code
//extract 1st digit
Digit1  $\leftarrow$  Code DIV 1000
Code  $\leftarrow$  Code - (1000 * Digit1)
//extract 2nd digit
digit2  $\leftarrow$  Code DIV 100
Code  $\leftarrow$  Code - (100 * Digit1)
//extract 3rd digit
Digit3  $\leftarrow$  Code DIV 10
Code  $\leftarrow$  Code - (10 * Digit1)
//extract 4th digit
Digit4  $\leftarrow$  DIV(Code, 1)
CheckDigit  $\leftarrow$  Code
//calculate check digit from data entered
//multiply by position and add together
Total  $\leftarrow$  (Digit1 * 4) + (Digit2 * 3) + (Digit2 * 2) + Digit3
NewCheckDigit  $\leftarrow$  MOD(Total, 11) //find Mod 11 of total
NewCheckDigit  $\leftarrow$  11 - NewCheckDigit //subtract result from
newCheckDigit
//check if the calculated check digit is the same as the last
digit in the //code
IF NewCheckDigit = CheckDigit THEN
    OUTPUT("Correct data entry")
ELSE
    OUTPUT("Incorrect data entry")
ENDIF
  
```


Verification of data

When entering data from another source, for example, a paper copy, it is important to make sure you have entered exactly what was written originally. This is called verification, a check that you have copied the data accurately.

There is a difference between entering data accurately, and entering the correct data. Verification only checks you have copied it accurately, the actual data may still be incorrect if the original was incorrect.

Two forms of verification are a visual check and double entry.

A visual check is where you compare the data entered to the original. For example, reading each line from a paper copy and checking it is identical on the computer.

A double entry check is where the same data is entered twice, usually by different people. The computer will then check whether there are any differences in what they each entered.

Checkpoint

1. What is validation?
2. Give three examples of validation checks.
3. What is the purpose of verification?
4. Identify which validation rule this algorithm uses: **INPUT Num**
IF Num > 0 AND Num < 100 THEN
OUTPUT "Valid"
ELSE
OUTPUT "Invalid"
ENDIF
5. Identify which validation rule this algorithm uses:
REPEAT
INPUT Value
UNTIL Value <> NULL AND Value <> ""
6. Write a validation routine to input a word and check that it is more than 10 characters long.
7. Write a validation routine to input a number and check that it is an integer value.

.....

.....

.....

.....

.....

.....

.....

.....

.....

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Trace tables

A trace table is a structure to help you follow an algorithm, for example, to find an error, or to work out what the algorithm does.

Each statement in an algorithm is run manually, and the values that are written to variables are written in the table in the order the changes are made.

Each column in the table is designated to one variable. There can also be a column for any outputs to be written in. User prompts are also entered in the trace table, these can be entered in a separate column or as an output (because they will be output). For example, `x = input("Enter a number")`, the text "Enter a number" is a user prompt and will need to be added to the trace table.

Specific inputs can be given to test different parts of the algorithm.

Trace table with selection

To create a trace table you need to identify all of the variables in an algorithm. Each variable should have its own column in the trace table. There should also be an output column if the program will output any data or strings.

For example, trace the following algorithm with the input values 1 and 3.

```
INPUT Number1  
INPUT Number2  
IF Number1 > Number2 THEN  
    OUTPUT(Number1)  
ELSE  
    OUTPUT(Number2)  
ENDIF
```

There are **two** variables and there is an output (each variable has a column, and the output has a column).

Number 1	Number 2	Number 3

You then complete this table by reading through the algorithm one line at a time. It is important not to jump ahead and guess what the algorithm will do. Read one line and perform that action.

Run line 1 of the code (input values 1, 3):

INPUT Number1

Number 1	Number 2	Number 3
1		

Run line 2 of the code (input values 1, 3):

INPUT Number2

Number 1	Number 2	Number 3
1	3	

Run the IF statement

```

IF Number1 > Number2 THEN
    ...
ELSE
    OUTPUT(Number2)
    
```

Number 1	Number 2	Number 3
1	3	3

Trace table with loops

When you are following an algorithm that includes a loop (iteration), some of the code will be repeated. To follow the code run one line at a time as before, and when you get to the end of the loop, go back to the start of the loop.

For example: trace the following algorithm with the input values 4, 3, 2, 1, 0.

```

01 Count←0
02 InputValue ← 1
03 Total←0
04 WHILE InputValue > 0 DO
05   INPUT InputValue
06   Total ← Total + InputValue
07   Count ← Count + 1
08 ENDWHILE
09 OUTPUT "The total is ", Total
10 OUTPUT "There were ", Count, " Numbers"
    
```

Start by creating the trace table with each of the variables and the output. There are 3 variables in this algorithm:

Count	Input Value	Total	Output

1. Run the first 3 lines of code. These are the initialisations, lines 01, 02 and 03.

01 Count ← 0

02 InputValue ← 1

03 Total ← 0

Count	Input Value	Total	Output
0	1	0	

2. Run line 04.

04 WHILE InputValue > 0 DO

LOOP is 1 so this is TRUE meaning that the code inside the loop is run

Run line 05 with the first input value of 4.

05 INPUT InputValue

Count	Input Value	Total	Output
0	1	0	
	4		

3. Run lines o6 and o7.

```
o6 Total←Total+ Total←o+4 InputValue
o7 Count←Count+1 Count←o+1
```

Count	Input Value	Total	Output
0	1	0	
1	4	4	

Line o8:

```
o8 ENDWHILE
```

sends you back to line o4.

```
o4 WHILE InputValue > o DO
```

InputValue is 4, so while 4 > o is TRUE to run line o5 with the second input value of 3.

```
o5 INPUT InputValue
```

Count	Input Value	Total	Output
0	1	0	
1	4	4	
	3		

5. Lines 06 and 07 run.

06 Total←Total+ InputValue Total←4+3
07 Count←Count+1 Count←1+1

Count	Input Value	Total	Output
0	1	0	
1	4	4	
2	3	7	

6. Line 08 ENDWHILE returns to line 04.

04 WHILE InputValue > 0 DO WHILE 3 > 0 is TRUE. Run lines 05, 06 and 07.
05 INPUT InputValue InputValue ← 2
06 Total←Total+ InputValue Total←7+2
07 Count←Count+1 Count←2+1

Count	Input Value	Total	Output
0	1	0	
1	4	4	
2	3	7	
3	2	9	

7. Line 08 again returns to line 04. WHILE 2 > 0 is TRUE. Run lines 05, 06 and 07.

05 INPUT InputValue InputValue ← 1
 06 Total ← Total + InputValue Total ← 9+1
 07 Count ← Count + 1 Count ← 3+1

Count	Input Value	Total	Output
0	1	0	
1	4	4	
2	3	7	
3	2	9	
4	1	10	

Line 08 returns to line 04 again. WHILE 1 > 0 is TRUE. Run lines 05, 06 and 07.

05 INPUT InputValue InputValue ← 0
 06 Total ← Total + InputValue Total ← 10+1
 07 Count ← Count + 1 Count ← 4+1

Count	Input Value	Total	Output
0	1	0	
1	4	4	
2	3	7	
3	2	9	
4	1	10	

9. Line 08 returns to line 04 again. WHILE 0 > 0 is FALSE this time, so the loop does not run.

Jump to line 09 run line 09 and 10.

09 OUTPUT "The total is ", Total

10 OUTPUT "There were ", Count, " numbers"

Count	Input Value	Total	Output
0	1	0	
1	4	4	
2	3	7	
3	2	9	
4	1	10	The total is 10
			There were 5 numbers

Finding the purpose of an algorithm

A trace table can help you work out the purpose of an algorithm. By following an algorithm step- by-step you can see what happens to the values, what changes are made, and when these are made. From this you can determine the purpose of the algorithm.

Finding the purpose with a trace table

Describe the processes in this algorithm and state its purpose.

```
INPUT Value1
INPUT Value2
INPUT Value3
IF Value1 > Value2 AND Value1 > Value3 THEN
    OUTPUT(Value1)
ELSEIF Value2 > Value1 AND Value2 > Value3 THEN
    OUTPUT(Value2)
ELSE
    OUTPUT(Value3)
ENDIF
OUTPUT(Value1 + Value2 + Value3)
```

- 1 . First test the algorithm with a set of data that you come up with (if they are not provided for you). For example, test this algorithm using 1, 2 then 3.
- 2 . Create a trace table by following the algorithm with the data input.

Value1	Value2	Value3	Output
1	2	3	3
			6

3. Repeat the process with a different set of data, for example, this time using 10, 5, 1.
4. Complete the trace table.

Value1	Value2	Value3	Output
10	5	1	10
			16

Keep on repeating this process until you can identify the pattern.

In this case it is outputting the largest number input and then outputting the total of the 3 numbers input.

Describing an algorithm

When you are describing an algorithm you need to do more than repeat the line of code using English statements. For example, in the following code, for line 02 don't just say, 'it is a for loop from y is 0 to x.' This is stating what each part of the code is, but it does not describe the algorithm.

```
01 INPUT X
02 FOR Y ← 0 TO X 03 OUTPUT Y
04 NEXT Y
```

You can do a trace table to help you work out the purpose of the algorithm before describing it if that helps you. This algorithm has three parts: line 01, line 02 and line 03. These are summarised below:

- Takes a number as input. (line 01)
- Loops from 0 to the number that has been input. (line 02)
- Outputs the values from 0 to the number. (line 03)

From this you can develop a description of the algorithm, for example:

The algorithm takes a number from the user, and then outputs all of the numbers from 0 to the number input.

Finding errors in a program and correcting the errors

To find an error in an algorithm you need to know the purpose of the algorithm. You can then test the algorithm with different data, completing trace tables if needed, to find out what the algorithm actually does. Once you know the difference between what it does and should do, you can start to identify the errors and the changes you need to make.

Using a trace table to find errors in a program

Using a trace table will allow you to follow each step in the algorithm and by comparing this to the intended outcome you can find where it goes wrong, and then work out how to correct it.

For example, this program should take 5 numbers as input and output the total of all the numbers.

```
01 Total ← 1
02 Quantity ← 1
03 WHILE Quantity < 5 DO
04   INPUT Number
05   Total ← Number
06   Quantity ← Quantity + 1
07 ENDWHILE
08 OUTPUT(Total)
```

First dry run the algorithm. If not provided, you will need to identify some test data to use. This algorithm should take 5 numbers as input, so you could use 1, 2, 3, 4, 5.

Then work out what the result of the algorithm should be. It outputs the total, so it should output: $1 + 2 + 3 + 4 + 5 = 15$

Now draw a trace table for the algorithm (revisit the section on trace tables if needed) and then run each step of the algorithm, writing the values in the table.

Total	Quantity	Number	Output
1	1		
1	2	1	
2	3	2	
3	4	3	
4	5	4	

Now look at the results and where it has gone wrong:

- 1 The total is overwritten each time, it should be added to.
- 2 Total starts with 1 but this will add another 1 to the total, so it should be 0.
- 3 Only four numbers can be input, it should be five.

Let's take each of the problems one at a time to solve them.

- 1 The total should not be overwritten. Line 05 overwrites the total. This should be **Total** $\leftarrow$ **Total + Number**.
- 2 The total should not start at 1, it should be 0. Line 01 writes 1 to total. This line should be **Total** $\leftarrow$ **0**.
- 3 Only 4 numbers are input, it should be 5. There are two ways this can be changed, both are valid. Either Line 02 can be **Quantity** $\leftarrow$ **0** or line 03 can be **WHILE Quantity < 6**.

Finding errors by reading the code

You may not always need to use a trace table, or you may not want to. There are many different ways of finding the errors. In this example you will look for each of the requirements in the algorithm to determine whether it has been met.

The following algorithm should take 3 numbers as input, and output the largest value and the average value.

```
01 Total ← 0
02 Largest ← 999
03 FOR X ← 0 TO 2
04   INPUT Data
05   IF Data < Largest THEN
06     Largest ← Data
07   ENDIF
08   Total ← Total + Data
09 NEXT X
10 OUTPUT Largest
11 Average ← Total / 3
12 OUTPUT Average
```

Re-read the description: take 3 numbers as input, and output the largest value and the average value.

This can be split into three sections:

1. Take 3 numbers as input.
2. Output the largest value.
3. Output the average value.

The two outputs need to be calculated before being output, so they can be added to the requirements:

1. Take 3 numbers as input.
2. Calculate the largest value.
3. Output the largest value.
4. Calculate the average value.
5. Output the average value.

Now check the first requirement:

Does it take 3 numbers as input?

The for loop goes from 0 to 2, so it will run 0, 1, 2 = 3 times. Inside the loop it inputs a value each time. This is correct.

The second requirement: Calculate the largest value.

To find the largest it needs to:

- A. Initialise a largest variable with a low number. This is not met. Line 02 largest is set to 999 but this should be a very small value, e.g. **Largest ← -1**
- B. It should check if the data input is larger than the data in largest. This is not met. Line 05 checks if data is less than 999. It should be larger. **IF Data > Largest THEN**
- C. Line 06 replaces the value in Large with data, but the identifier is incorrect, the variable is named largest. **Largest ← Data**

The third requirement:

Output the largest value.

Line 10 outputs the largest value, **this is correct**.

The fourth requirement:

Calculate the average value.

The average is calculated by totalling the inputs and then dividing by the number of inputs. Line 08 totals the data entered, but it uses the **wrong identifier**, it should be data not DATA.

Total ← Total + Data

Line 11 works out the average **correctly**. The fifth requirement:

Output the average value.

Line 12 outputs the average **correctly**.

In this case the errors and corrections have been identified along the way. You can also combine this with using test data to check both the original algorithm and to check that your corrections are accurate.